S-LINK 2.0 規格書・機能表

文書の版 0.1（DRAFT） ／ 発行日 2026-10-27

生成元

ファイル	版
`S-LINK2.0/S-LINK2.0-規格書.md`	文書の版 0.1（タグ `v0.1`）
`S-LINK2.0/S-LINK2.0-機能表.md`	文書の版 0.1（タグ `v0.1`）

リポジトリ: `https://github.com/s-link/s-link-spec`

DRAFT

S-LINK 2.0 規格書

項目	内容
文書の版	0.1（DRAFT）
発行日	2026-10-27
対象プロトコル	S-LINK 2.0（Protocol ID `0x02`、`version_minor` 0）
対応する機能表	版 1

文書の版は、この規格書の文章の改訂を示す番号です。通信上の版番号（`version_minor`、`function_table_revision`）とは独立しています。改訂の内容は末尾の「改訂履歴」に記載します。

1 適用範囲

この規格では、ホストが計測制御機器からセンサー値を読み取り、機器の出力を制御するためのメッセージ形式と通信手順を定めます。

この規格で定めるのは、次の 2 点です。

- メッセージのバイト構成
- ホストとデバイスの動作

メッセージの搬送には USB-MIDI 1.0 を使い、その上で MIDI 1.0 のシステム・エクスクルーシブ（System Exclusive、SysEx）メッセージを送受信します。USB-MIDI の転送形式と MIDI のメッセージ構文は、「2 引用規格」に記載した規格に従います。

1.1 全体の見取り図

通信の全体像を示すため、S-LINK を次の 4 層に分けて説明します。この規格が定めるのは第 2 層で、第 3 層の機能定義は機能表に示します。

層	内容	定める文書
第 4 層（アプリケーション）	アプリケーションとの連携、画面設計	各アプリケーションの仕様
第 3 層（機能定義）	機能番号、単位、データ形式	機能表
第 2 層（メッセージ）	コマンドとバイト構成	この規格
第 1 層（搬送）	USB-MIDI 1.0 による MIDI 1.0 SysEx の転送	「2 引用規格」に記載した規格

各開発者は第 4 層のアプリケーションを独自に開発できます。同じ機器を使うアプリケーションは切り替えられますが、同時に操作するホストは 1 つです。1 つのホストから複数台を扱う接続は、1.2 に定めます。

DRAFT

1.2 1 ホストと複数デバイスの接続

1 つのホストは、USB ハブを通じて接続した複数の S-LINK デバイスを同時に扱えます。異なるメーカーの機器でも、同一機種 2 台でも、OS が機器ごとに別の MIDI 入出力ポートを公開していれば区別できます。接続ごとに 1 組の MIDI 入出力ポートを使います。

```
1 つのブラウザーアプリ
  ├─ MIDI out A / in A ─ USB ハブ ─ A 社のデバイス
  └─ MIDI out B / in B ─ USB ハブ ─ B 社のデバイス
```

この図の A / B はホスト側の接続の区別です。メッセージの `port` は各デバイス内の割り当て先であり、USB ポート番号、MIDI ポート、USB-MIDI の Cable Number とは異なります。

- ホストは 5.4 の `HELLO` で MIDI 入出力ポートを対応付け、接続ごとに管理する。ポート名や `model_id` が同じでも接続をまとめない
- 受信途中の SysEx、`transaction_id` と応答待ち、タイムアウト、`CAPABILITY`、割り当て先の状態、通知登録とイベント番号、通信途絶監視を接続ごとに管理する。応答・通知は受信元の MIDI 入力ポートに属するものとして照合する
- 出力または通知登録を維持する接続それぞれに、1 秒以内ごとにリクエストを送る。別の機器への送信では、その接続の維持にはならない
- 機器の追加時は、新しい接続で探索と初期化を行う。取り外された接続では、応答待ちと通知登録を終了し、古い受信データを破棄する。ほかの接続は継続する。再接続時は改めて `HELLO` から初期化する

「1 台のデバイスを同時に操作するホストは 1 つ」という制限は、ホストが扱えるデバイスの台数を 1 台に制限するものではありません。複数ホストが同じデバイスを操作する場合の調停は定めません。

同一機種 2 台は、接続中は MIDI 入出力ポートの組で区別します。`model_id` は機種を示し、`device_id` は個体を示します。`DEVICE_ID` に対応する機器では、ホストは問い合わせで得た `device_id` を保存して抜き差し後の識別に使えます。`DEVICE_ID` に対応しない機器については、抜き差し後の個体識別を保証しません。

USB ハブは、1 回の MIDI 送信をすべての機器へ配信するものではありません。この規格は、別々の MIDI ポートを通じた個別の通信を対象とします。同じ操作を複数台へ行う場合も、ホストは接続ごとにリクエストを送り、応答を個別に扱います。同一メッセージのブロードキャスト、複数機器の MIDI ストリームの合流、宛先 ID による振り分けや複数台の同時実行保証は定めません。

ブラウザーでの利用例は、附属書 B を参照してください。

2 引用規格

USB-MIDI の転送形式と MIDI メッセージの構文には、次の版を適用します。外部規格の改訂を自動で取り込むものではありません。

DRAFT

文書	版・発行元	この規格で参照する内容
Universal Serial Bus Device Class Definition for MIDI Devices	Release 1.0、1999-11-01、USB-IF	箇条 4 の USB-MIDI Event Packet、Cable Number、Code Index Number（CIN）と、MIDIStreaming の転送形式
MIDI 1.0 Detailed Specification	Document Version 4.2.1、1996-02 改訂、MIDI Manufacturers Association	MIDI 1.0 のデータバイト、System Exclusive、Manufacturer ID、System Real Time の構文。DIN の物理接続は適用範囲に含めない
S-LINK 2.0 機能表	`HELLO` の `function_table_revision` で示す版	機能番号、単位、値の形式と動作

USB-MIDI は SysEx を USB 上で運ぶ方法、MIDI 1.0 は SysEx 自体の構文を定めます。USB-MIDI Event Packet のヘッダーや未使用バイトは、S-LINK メッセージの一部ではありません。

3 用語及び定義

用語	定義
ホスト	リクエストを送る側。PC、Chromebook、タブレットのアプリケーション
デバイス	リクエストにレスポンスを返す側。計測制御機器
MIDI ポート	OS がホストに公開する MIDI の入力先または出力先。デバイス内の `port` とは別のもの
接続	`HELLO` で対応付けた MIDI 入出力ポートの組と、その組を通じた 1 台のデバイスとの通信
`port`	割り当て先の番号。`0x00`–`0x7E` を割り当てに使い、`0x7F` は対象がないことを示す
チャンネル	`function_id` と `port` の組。入出力の 1 単位
入力機能	センサーなどから値を取得する機能。`access` が `read`
出力機能	モーターや LED などを駆動する機能。`access` が `write`
入力チャンネル	入力機能を持つチャンネル。ホストが `READ` で値を読み取る
出力チャンネル	出力機能を持つチャンネル。ホストが `WRITE` で値を書き込む
端子	機器の外に出ている物理の接続点
割り当て先	`port` が示す、機能を利用する対象。物理端子や機器内の機能を表す
入力状態 / 出力状態 / 解放状態	デバイスが `port` ごとに管理する状態
入力状態	デバイスがその割り当て先を入力として扱う状態
出力状態	デバイスがその割り当て先を出力として扱う状態。実際に駆動できているかは機能の動作による
解放状態	その割り当て先を駆動していない状態。電源投入後の初期状態

DRAFT

用語	定義
チャンネルディスクリプター	`CAPABILITY` のレスポンスに含まれる、1 チャンネル分の情報。長さは `descriptor_bytes` が示す。USB のディスクリプターとは別物
リクエスト	ホストがデバイスへ送るメッセージ
レスポンス	デバイスがホストへ返すメッセージ。`ERROR` と `PENDING` を含む
正常レスポンス	リクエストが成功したときのレスポンス。`HELLO` / `MODEL_ID` / `DEVICE_ID` / `FIRMWARE_VERSION` / `CAPABILITY` / `PORT_NAME` / `KEEPALIVE` / `READ` / `INPUT_SUBSCRIBE` / `INPUT_UNSUBSCRIBE` は同じコマンド番号で返し、`WRITE` と `PORT_RELEASE` は `ACK` で返す

ここでいう入力・出力は、デバイスを基準にした呼び方です。

メッセージの方向は、`H→D`（ホストからデバイス）または `D→H`（デバイスからホスト）で表します。

---

4 データ表現

4.1 メッセージ形式

S-LINK 2.0 のメッセージ形式を以下に示します。位置 `off` は、System Real Time メッセージを取り除いた SysEx の先頭 `F0` を 0 として数えます。

```
F0 00 40 08 02 <コマンド> <payload_length> <ペイロード...> F7
```

位置	内容	型・長さ
`off 0`	`F0`（SysEx 開始）	1 byte
`off 1–3`	`00 40 08`（System Exclusive Manufacturer ID）	3 byte
`off 4`	プロトコル識別子（Protocol ID）	U7
`off 5`	コマンド番号	U7
`off 6`	`payload_length`	U7。ペイロードの byte 数。0〜66
`off 7–6 + payload_length`	ペイロード。長さ 0 の場合はこの部分なし	`payload_length` byte
`off 7 + payload_length`	`F7`（End of Exclusive）	1 byte

`00 40 08` は、一般社団法人音楽電子事業協会（AMEI）により、株式会社ティーファブワークスに割り当てられた System Exclusive Manufacturer ID です。この ID は、この規格に準拠する S-LINK メッセージに使用できます。

長さの数え方

- `payload_length` は、`off 7` から `F7` の直前までの符号化後のデータバイト数。`transaction_id` など、各コマンドのフィールドをすべて含む。`INPUT_EVENT` では `subscription_id` から数える
- `F0`、Manufacturer ID、Protocol ID、コマンド番号、`payload_length` 自身、`F7` はペイロード長に含めない。System Real Time、USB-MIDI Event Packet のヘッダーと未使用バイトも含めない
- メッセージ全長は `8 + payload_length` byte。ペイロード上限は 66 byte、全長上限は 74 byte。`U7` で表せる 67〜127 は、このフィールドでは無効。0 は枠として表現できるが、現行コマンドの必須フィールドは省略できない
- ASCII は 1 文字を 1 byte、TEXT は UTF-8 の 1 byte を 2 byte に符号化して数える。機能表の `payload` 列は値の部分だけを表し、このフィールドとは数える範囲が異なる
- `F0` と `F7` の間のデータバイトは 7 bit とし、最上位ビット（MSB）は 0。使用する型は `U7` / `U14` / `U28` / `U42` / `S28` / `S7` / `ASCII` / `TEXT` とする

受信処理

1. USB-MIDI Event Packet から CIN に従って有効な MIDI バイトを取り出し、ヘッダーと未使用バイトを除く。USB 転送や API の受信単位を SysEx の境界とみなさず、接続ごとに再構成する
2. System Real Time のステータス（`0xF8`–`0xFF`。未定義の `0xF9` / `0xFD` は無視）を、SysEx の組み立てや長さの計数より先に切り分ける。別の MIDI 機能で処理するか無視し、SysEx の受信位置と長さは変えない。ヘッダー受信中や読み捨て中も同じ扱いとする
3. Manufacturer ID と Protocol ID を確認する。対象外の SysEx は S-LINK として解釈せず、終端まで読み飛ばす。別の MIDI 機能で処理することは妨げない
4. `payload_length` を読み、実際のペイロード長と照合する。宣言した長さに達しただけではコマンドを実行せず、`F7` まで受信して一致を確認する。各コマンドの必須フィールドと値の長さも別に確認する。将来の版による未知の末尾フィールドも、受信した `payload_length` の計数には含める
5. 長さフィールドがない、長さが一致しない、`payload_length > 66`、または実際のペイロードが 66 byte を超えた場合、そのメッセージを実行しない。上限超過や宣言長より多いデータを検出した後は、追加データを蓄積せず `F7` まで読み捨てる

`F7` で終了した不正なメッセージに対し、デバイスは Manufacturer ID と Protocol ID `0x02`、受信コマンドがリクエストであること、実際に受信した `off 7`–`8` の `transaction_id` を確認できる場合に限り、`ERROR(BAD_REQUEST)` を 1 回返します。確認できない場合は応答しません。ホストは不正なレスポンスや通知を破棄し、それへの `ERROR` は送りません。応答待ちは、正しいレスポンスを受けるか期限を過ぎるまで継続します。不正なメッセージで出力・通知登録を変更したり、通信途絶の監視時間を更新したりしません。

System Real Time 以外のステータスが `F7` より先に現れた場合、途中の SysEx は破棄して応答せず、そのステータスから MIDI の受信処理を再開します。新しい `F0` なら新たな SysEx の開始です。読み捨て中も同様です。切断時は組み立て途中のデータを破棄します。

System Real Time が間に入ると、伝送上の `F0` から `F7` までのバイト数が 74 を超えることがあります。それだけでは長さ違反ではありません。上限は、System Real Time を除いた S-LINK メッセージに適用します。

バイト列の例（16 進数）

DRAFT

```
HELLO リクエスト、transaction_id = 1
F0 00 40 08 02 00 02 01 00 F7
                  ↑  ↑——— ペイロード 2 byte
payload_length = 2、全長 10 byte

HELLO レスポンス、transaction_id = 1、version_minor = 0、機能表の版 = 1
F0 00 40 08 02 00 05 01 00 00 01 00 F7
payload_length = 5、全長 13 byte

HELLO リクエストの途中に Timing Clock（F8）が入った受信ストリーム
F0 00 40 08 02 00 02 01 F8 00 F7
F8 を切り分けると、上の HELLO リクエストと同じ 10 byte

USB-MIDI Event Packet にした HELLO リクエスト（Cable Number = 0）
04 F0 00 40 | 04 08 02 00 | 04 02 01 00 | 05 F7 00 00
各組の先頭は Cable Number / CIN。最後の 00 00 は未使用で、長さに含めない
```

4.2 プロトコル識別子

`off 1–3` の Manufacturer ID と、`off 4` の Protocol ID を組み合わせてメッセージ形式を識別します。S-LINK 2.0 には `0x02` を用い、マイナー版は `HELLO` の `version_minor` で示します。Protocol ID は、`off 5` のコマンド番号とは別です。

デバイスは、知らないプロトコル識別子のメッセージに応答せず、破棄します（箇条 9 の 1）。4.1 の長さフィールドも `0x02` だけに適用します。

4.3 データ型と符号化

7 bit を超える値は複数のバイトに分けて表します。複数バイトの値は、最下位バイト（LSB）から順に送信します。

型	byte 数	範囲
`U7`	1	0 ～ 127
`S7`	1	−64 ～ 63（2 の補数）
`ASCII`	1	表示可能 ASCII（`0x20–0x7E`）
`U14`	2	0 ～ 16,383
`U28`	4	0 ～ 268,435,455
`U42`	6	0 ～ 4,398,046,511,103
`S28`	4	−134,217,728 ～ 134,217,727（2 の補数）
`TEXT`	2n	UTF-8 の n byte（符号化前）。1 byte を 2 byte に分けて送信する

DRAFT

```
送る側が U14 の 5000 を組み立てる
  下位 7 bit  5000 & 0x7F        = 0x08
  上位 7 bit  (5000 >> 7) & 0x7F = 0x27
  バイト列    08 27

受け取る側が U14 を復元する
  値 = b0 | (b1 << 7)
```

```
S28 の 5000 と −5000
  5000     08 27 00 00
  −5000    78 58 7F 7F

受け取る側が S28 を復元する
  v = b0 | (b1 << 7) | (b2 << 14) | (b3 << 21)
  v ≥ 0x8000000 のとき   v − 0x10000000

送る側が S28 の −5000 を組み立てる
  v < 0 のとき           v + 0x10000000 = 268,430,456
  以後は U28 と同じく 7 bit ずつ 4 バイトに分ける
  バイト列               78 58 7F 7F

受け取る側が S7 を復元する
  v = b0
  v ≥ 0x40 のとき        v − 0x80

送る側が S7 の −2 を組み立てる
  v < 0 のとき           v + 0x80 = 126
  バイト列               7E
```

- `U28` は 7 bit ずつ 4 バイト
- `U42` は 7 bit ずつ 6 バイト
- `S28` は 28 bit の 2 の補数。符号ビットは 4 バイト目の bit6
- `S7` は 7 bit の 2 の補数。符号ビットは bit6
- `ASCII` は 1 byte のまま送信する。`TEXT` は UTF-8 の 1 byte を 2 byte に分けて送信する

`ASCII` に含められるのは、表に示した表示可能文字です。制御文字と `0x7F` は含めません。

`U42` は、下位 7 bit を先頭にして次の式で復元します。

```
v = b0 | (b1 << 7) | (b2 << 14) | (b3 << 21) | (b4 << 28) | (b5 << 35)

U42 の 1500   →   5C 0B 00 00 00 00
```

符号付きの値には、必要な範囲と分解能を表せる型を使います。たとえば ±100 % を 0.01 % 単位で表すには整数値が ±10,000 となるため、符号付き 14 bit（−8,192〜8,191）には収まりません。この値域には `S28` を使います。

```
送る側が TEXT を組み立てる
  1 byte につき  下位 4 bit  b & 0x0F          を先に
                 上位 4 bit  (b >> 4) & 0x0F   を後に

受け取る側が TEXT を復元する
  b = n0 | (n1 << 4)

「明」 E6 98 8E   →   06 0E   08 09   0E 08
「A1」 41 31      →   01 04   01 03
```

- 測定値の型は `function_id` ごとに機能表が定める
- RGB の各成分は `U7`（0–127）

---

5 コマンド

5.1 コマンド一覧

番号	コマンド名	方向	用途	ペイロード長	全長
`0x00`	`HELLO`	H→D / D→H	相手が S-LINK 2.0 の機器かを確かめる	2 / 5	10 / 13
`0x01`	`MODEL_ID`	H→D / D→H	機種を特定する	2 / 2 + n	10 / 10 + n（1 ≤ n ≤ 64）
`0x02`	`DEVICE_ID`	H→D / D→H	個体を識別する固定 ID を問い合わせる	2 / 2 + n	10 / 10 + n（1 ≤ n ≤ 64）
`0x03`	`FIRMWARE_VERSION`	H→D / D→H	ファームウェアの版を問い合わせる	2 / 2 + n	10 / 10 + n（1 ≤ n ≤ 64）
`0x04`	`CAPABILITY`	H→D / D→H	デバイスが対応するチャンネルの一覧を問い合わせる	3 / 5 + 14n	11 / 13 + 14n（0 ≤ n ≤ 4）
`0x05`	`PORT_NAME`	H→D / D→H	割り当て先の名前を問い合わせる	3 / 3 + 2n	11 / 11 + 2n（0 ≤ n ≤ 30）
`0x06`	`KEEPALIVE`	H→D / D→H	ホストが動作中であることを伝える	2 / 2	10 / 10
`0x07`	`ACK`	D→H	リクエストが成功したことを返す	6	14
`0x08`	`ERROR`	D→H	リクエストが失敗した理由を返す	7	15
`0x09`	`PENDING`	D→H	処理中であることと、待つ時間を伝える	8	16

DRAFT

番号	コマンド名	方向	用途	ペイロード長	全長
0x10	READ	H→D / D→H	値を 1 つ読み取る	5 / 5 + d	13 / 13 + d
0x11	INPUT_SUBSCRIBE	H→D / D→H	入力チャンネルの状態変化通知を開始する	11 / 22	19 / 30
0x12	INPUT_UNSUBSCRIBE	H→D / D→H	状態変化通知を終了する	11 / 11	19 / 19
0x13	INPUT_EVENT	D→H	入力チャンネルの状態変化を通知する	20	28
0x20	WRITE	H→D	出力を駆動する	5 + d	13 + d
0x21	PORT_RELEASE	H→D	割り当て先の駆動を止め、解放状態に戻す	3	11

長さの単位は byte です。`H→D / D→H` の順に示します。`n` は各行の文字列の符号化前の byte 数またはディスクリプターの個数、`d` は機能の値の符号化後の byte 数（上限 61）です。全長は `F0` と `F7` を含み、System Real Time を含みません。`CAPABILITY` で返す個数の条件は 5.8 に従います。

`HELLO` / `MODEL_ID` / `DEVICE_ID` / `FIRMWARE_VERSION` / `CAPABILITY` / `PORT_NAME` / `KEEPALIVE` / `READ` / `INPUT_SUBSCRIBE` / `INPUT_UNSUBSCRIBE` は、リクエストとレスポンスで同じコマンド番号を用います。 方向は、どちらが送ったかで決まります。 `WRITE` と `PORT_RELEASE` の正常レスポンスは `ACK` です。 リクエストが失敗したときのレスポンスは `ERROR` です。 `INPUT_EVENT` はリクエストへの応答ではありません。

5.2 コマンド番号の割り当て

範囲	用途	割り当て済み	空き
0x00–0x0F	接続、機器情報、共通応答	0x00–0x09	0x0A–0x0F
0x10–0x1F	入力値の読み取りと状態変化通知	0x10–0x13	0x14–0x1F
0x20–0x2F	書き込みと割り当て先の操作	0x20 0x21	0x22–0x2F
0x30–0x7F	予備	—	全部

未割り当ての番号は、同じ用途のコマンドを追加するために確保しています。

5.3 共通のフィールド

`transaction_id` / `function_id` / `port` は複数のメッセージに現れます。 バイト位置と型は、各メッセージの表によります。各表の `off` は SysEx 先頭からの絶対位置です。共通ヘッダーの `payload_length`（`off 6`、U7）はすべてのコマンドに含め、5.1 のペイロード長を設定します。各表では共通ヘッダーと末尾の `F7` を省略します。チャンネルディスクリプターの表だけは、ディスクリプター先頭からの相対位置です。

DRAFT

フィールド名	型	内容
`transaction_id`	U14	リクエスト番号。ホストはレスポンスとの対応付けに使い、デバイスは同じ値を返す
`function_id`	U14	機能番号。機能表が定める番号。`0x3FFF` は「該当なし」
`port`	U7	チャンネルの割り当て先を表す番号。`0x00`–`0x7E` を割り当てに使い、`0x7F` は「該当なし」

- 同じ接続内で同じ値の `port` は、同じ割り当て先を指す。別のデバイスの同じ番号とは区別する

5.4 `HELLO`（`0x00`）

ホストは、接続先が S-LINK 2.0 の機器かどうかを確かめます。レスポンスを返すのは S-LINK 2.0 の機器だけです。

リクエスト

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストごとに変える番号

レスポンス

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストの値をそのまま返す
9	`version_minor`	U7	S-LINK のマイナー版番号。この規格では `0`。プロトコル識別子 `0x02` と組み合わせて S-LINK 2.0 を示す
10-11	`function_table_revision`	U14	デバイスが準拠する機能表の版

ホストは探索中の各 MIDI out ポートから、探索全体で異なる `transaction_id` を付けて `HELLO` を送ります。すべての候補 in ポートで応答を待ち、受信元の in ポートと `transaction_id` から out ポートとの組を確定します。探索中・再利用待ちの番号は別の探索に使いません。一つの探索要求に複数の in ポートから応答するなど、組を一意に決められない場合は接続を確定せず、MIDI の複製・合流などの経路を確認します。

組の確定後は、接続ごとに `transaction_id` を管理します。別の接続で同じ番号を使っても、受信元ポートで区別します。`subscription_id` も接続ごとに管理します。

この規格より大きい `version_minor` を受け取ったホストは、認識できるコマンドだけを使います。認識できないコマンドやフィールドは無視します。

同じプロトコル識別子の範囲では、既存コマンドのリクエスト形式、フィールドの位置と意味を変更しません。新しいリクエスト形式が必要な場合は、新しいコマンドを割り当てます。既知のレスポンスに未知の末

DRAFT

尾フィールドがある場合、ホストはそれを無視します。

S-LINK は、1 台のデバイスを同時に操作するホストが 1 つであることを前提とします。複数のアプリケーションが同じデバイスを同時に操作する場合の調停方法は定めません。 ブラウザーのタブやアプリケーションが複数ある場合、それぞれが別のホストとして動作します。同時に操作すると、`transaction_id` の衝突で応答を取り違えたり、一方の `KEEPALIVE` が他方の出力を維持したりすることがあります。

```
out 1 に HELLO(transaction_id=1)
out 2 に HELLO(transaction_id=2)      →  transaction_id=2 が返ってきた in が out 2 と組になる
out 3 に HELLO(transaction_id=3)
```

5.5 `MODEL_ID`（`0x01`）

ホストは、デバイスの機種を特定します。

リクエスト

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストごとに変える番号

レスポンス

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストの値をそのまま返す
9-	`model_id`	ASCII	機種の識別子。1〜64 byte

- `model_id` には、識別子の重複を避けるため逆ドメイン形式を使う（例 `com.tfabworks.tfw-at1`）。ホストは値の意味を解釈せず、文字列として照合する
- `model_id` は機種の判別に使う。機種名の表示や、その機種に固有の処理を選ぶときに利用する
- `model_id` を取得できない場合も、値の読み書きはできる。チャンネル一覧は `CAPABILITY` から取得し、単位と表示名は機能表で確認する
- デバイスは `MODEL_ID` の実装を省いてよい。省いたデバイスは `ERROR(UNSUPPORTED_COMMAND)` を返す

5.6 `DEVICE_ID`（`0x02`）

ホストは、機器の個体を識別する ID を問い合わせます。`MODEL_ID` が機種を表すのに対し、`DEVICE_ID` は同じ機種の機器同士を区別します。接続中は MIDI 入出力ポートの組で機器を区別し、`DEVICE_ID` は主に抜き差し後に同じ個体を見分けるために使います。

リクエスト

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストごとに変える番号

DRAFT

レスポンス

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストの値をそのまま返す
9-	`device_id`	ASCII	個体ごとに固定した ID。1〜64 byte

- `device_id` は個体ごとに異なり、電源の再投入やファームウェア更新後も変わらない値とします。機器の製造時に生成したランダム値や、MPU などの変更されない固有 ID を使えます
- 数値やバイナリーの固有 ID を使う場合も、16 進数などの表示可能 ASCII 文字列にして返します。ホストは値の意味を解釈せず、文字列として照合します
- `device_id` は `model_id` や製品の型番ではありません。氏名など個人を表す情報を含めず、製造番号を兼ねる必要もありません
- `HELLO` では送信しません。ホストが必要に応じて問い合わせます
- デバイスは `DEVICE_ID` の実装を省いてよいものとします。省いたデバイスは `ERROR(UNSUPPORTED_COMMAND)` を返します。未対応機器について、ホストは再接続後の個体識別を保証できません

```
DEVICE_ID リクエスト、transaction_id = 2
F0 00 40 08 02 02 02 02 00 F7
payload_length = 2、全長 10 byte

DEVICE_ID レスポンス、transaction_id = 2、device_id = "A1"（例）
F0 00 40 08 02 02 04 02 00 41 31 F7
payload_length = 4、全長 12 byte。ASCII なので 41 31 をそのまま送る
```

5.7 `FIRMWARE_VERSION`（`0x03`）

ホストは、デバイスで動作しているファームウェアの版を問い合わせます。

リクエスト

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストごとに変える番号

レスポンス

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストの値をそのまま返す
9-	`firmware_version`	ASCII	ファームウェアの版。1〜64 byte

- 版の付け方は機器の開発者が決める（例 `1.2.3`、`2026.09.29`）
- ホストは画面表示や診断ログに使う。文字列の大小で版の新旧や対応機能を判断しない。対応するチャンネルは `CAPABILITY` で確認する

- デバイスは `FIRMWARE_VERSION` の実装を省いてよい。省いたデバイスは `ERROR(UNSUPPORTED_COMMAND)` を返す

5.8 `CAPABILITY`（`0x04`）

ホストは `CAPABILITY` で、デバイスが対応するチャンネルを問い合わせます。 `READ` と `WRITE` で指定できるのは、レスポンスに含まれる `(function_id, port)` の組です。

チャンネルは `function_id` と `port` の組で表します。同じ割り当て先に複数のチャンネルを割り当てることもできます。次は、機器が宣言するチャンネルの例です。

```
(ILLUM,      port 0x06)     明るさセンサー
(TEMP,       port 0x07)     温度センサー
(ANALOG_IN,  port 0x00)     生の A/D 値
(SERVO,      port 0x04)     サーボモーター出力
(DIGITAL_IN, port 0x00)     スイッチ入力
```

- デバイスは、対応する入力チャンネルと出力チャンネルをレスポンスで返す
- チャンネルがない機器（`total = 0`）は、 `index = 0` のリクエストに対して、チャンネルディスクリプターを含まないレスポンスを返す
- 外付けセンサーが接続されていなくても、デバイスは対応するチャンネルを宣言する
- 1 回に返せるのは `floor((66 − 5) / descriptor_bytes)` チャンネルまで。この版では最大 4 チャンネル

リクエスト

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストごとに変える番号
9	`index`	U7	何番目のチャンネルから返すか。0 起算

レスポンス

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストの値をそのまま返す
9	`total`	U7	この機器が対応するチャンネルの総数
10	`index`	U7	このメッセージの先頭が何番目か。リクエストの値をそのまま返す
11	`descriptor_bytes`	U7	チャンネルディスクリプター 1 個の byte 数。この版では 14
12-	チャンネルディスクリプター	—	`descriptor_bytes` byte ずつ、最大 `floor((66 − 5) / descriptor_bytes)` 個

`total` は U7 で表すため、チャンネル数の上限は 127 です。`total = 0` の場合は `index = 0` に限って空のレスポンスを返します。それ以外では、 `index` が `total` 以上なら `ERROR(BAD_REQUEST)` を返します。

DRAFT

`total > 0` で `index < total` の場合、デバイスは `index` から順に、少なくとも 1 個のチャンネルディスクリプターを返します。返す個数を `n` とすると、`1 ≤ n ≤ min(total − index, floor((66 − 5) / descriptor_bytes))` とします。残りのチャンネル数を超えて返すことはできません。

ホストは、ディスクリプター部分の長さが `descriptor_bytes` の整数倍でない場合や、返された個数が上記の条件を満たさない場合、そのレスポンスを不正として破棄します。

デバイスは予約ビット（`flags` の bit3–6）を 0 にします。ホストは受信した予約ビットを無視します。

```
total = 15 の機器
  CAPABILITY(index=0)   →  total=15, index=0,  チャンネルディスクリプター 4 個
  CAPABILITY(index=4)   →  total=15, index=4,  チャンネルディスクリプター 4 個
  CAPABILITY(index=8)   →  total=15, index=8,  チャンネルディスクリプター 4 個
  CAPABILITY(index=12)  →  total=15, index=12, チャンネルディスクリプター 3 個
```

チャンネルディスクリプター（14 byte）

off	フィールド名	型	意味
0-1	`function_id`	U14	機能の識別子
2	`port`	U7	チャンネルの割り当て先を表す番号。`0x00`–`0x7E` を割り当てに使う
3	`flags`	U7	bit0 = 0 入力機能 / 1 出力機能。bit1 = 1 のとき `pull_mv` / `pull_ohm` が有効。bit2 = 1 のとき、機能表の説明欄で定める 0 と最大値だけを受け付ける。負の値を持つ機能では bit2 を 1 にしない。bit3–6 予約
4	`bits`	U7	アナログ・デジタル変換の分解能。範囲 0–28。該当する値がない場合 0
5-6	`reference_mv`	U14	アナログ・デジタル変換の基準電圧。0–16,383 mV。該当する値がない場合 0
7	`exponent`	S7	値 × 10^exponent が物理量。`kind: quantity` 以外は 0
8-9	`pull_mv`	U14	外部負荷を接続しないときに端子に現れる電圧。0–16,383 mV。物理端子以外では 0
10-13	`pull_ohm`	U28	端子から内部回路側を見たテブナン等価抵抗。単位 Ω。物理端子以外では 0

`bits` と `reference_mv` は、`kind: adc` の機能で値を ADC 入力電圧に直すために用います。`adc` は A/D 値（アナログ・デジタル変換で得た値）を表します。分圧回路がある場合も、この式が求めるのは ADC 入力の電圧です。端子側の電圧へ換算するには分圧比が別途必要です。`pull_mv` と `pull_ohm` だけから分圧比は求められません。 `bits = 0` の場合、この式による電圧換算は行いません。

DRAFT

```
電圧[mV] = 値 × reference_mv / (2^bits − 1)

bits = 12、reference_mv = 3300 のとき
  値 2048  →  2048 × 3300 / 4095 = 1650.4 mV
```

`reference_mv` は、アナログ・デジタル変換で得られる最大値に対応する電圧です。たとえば `bits = 12` の場合、最大値 4095 が `reference_mv` を表します。

5.9 `PORT_NAME`（`0x05`）

ホストは `PORT_NAME` で割り当て先の名前を問い合わせます。デバイスは、画面表示に使える名前を返します。

リクエスト

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストごとに変える番号
9	`port`	U7	名前を取得する割り当て先

レスポンス

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストの値をそのまま返す
9	`port`	U7	リクエストの値をそのまま返す
10-	`name`	TEXT	割り当て先の名前。UTF-8 で 30 byte 以内

次の例は、`PORT_NAME` が返す割り当て先の名前です。名前は機器ごとに異なります。

```
PORT_NAME(port 0x00)   →  左端子
PORT_NAME(port 0x04)   →  サーボモーター接続先
PORT_NAME(port 0x06)   →  明るさ
```

- 名前は `port` ごとに 1 つです。同じ割り当て先に対応するチャンネルには、同じ名前を使います
- ホストは `CAPABILITY` で取得した各 `port` について、`PORT_NAME` を 1 回送ります。同じ `port` のチャンネルが複数あっても、問い合わせは 1 回で足ります
- `PORT_NAME` は省略できます。実装しないデバイスは `ERROR(UNSUPPORTED_COMMAND)` を返します
- 名前を取得できない割り当て先は、ホストが `port` 番号を画面に表示します

名前はデバイスから取得するため、ホスト側で機種ごとの割り当て先名を管理する必要はありません。

5.10 `KEEPALIVE`（`0x06`）

`KEEPALIVE` は、ホストが動作中であることをデバイスに知らせるメッセージです。ほかに送るリクエストがない場合に使います。

リクエスト

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストごとに変える番号

レスポンス

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストの値をそのまま返す

- 端子や出力の状態は変わりません。全長は 10 byte で、規格内のメッセージでは最短です
- ホストは `KEEPALIVE` 以外のリクエストを送って、動作中であることを知らせてもかまいません
- デバイスからレスポンスが返るため、ホストはデバイスの動作も確認できます

ホストの送信方法の例を示します。定期的に `READ` を送るホストは、その応答で動作を確認できます。`READ` を行わず `WRITE` で出力を制御するホストは、`KEEPALIVE` を定期的に送ります。

```
READ を定期送信するホスト    READ の応答が動作確認を兼ねる。KEEPALIVE は不要
WRITE で出力を制御するホスト    500 ms ごとに KEEPALIVE を送る
```

5.11 `ACK`（`0x07`）

デバイスは、`WRITE` または `PORT_RELEASE` の成功を通知します。

off	フィールド名	型	意味
7-8	`transaction_id`	U14	成功したリクエストに付いていた番号
9	`request_command`	U7	成功したリクエストのコマンド番号
10-11	`function_id`	U14	成功したリクエストの `function_id`。`PORT_RELEASE` では `0x3FFF`
12	`port`	U7	成功したリクエストの `port`

ホストは、リクエストごとに `ACK` を待つ方法と、複数のリクエストを送ってから `transaction_id` で `ACK` と対応付ける方法のどちらを使ってもかまいません。

5.12 `ERROR`（`0x08`）

`ERROR` は、リクエストが失敗した理由を示します。

off	フィールド名	型	意味
7-8	`transaction_id`	U14	失敗したリクエストに付いていた番号
9	`request_command`	U7	失敗したリクエストのコマンド番号
10-11	`function_id`	U14	失敗したリクエストの `function_id`。この欄がないリクエストでは `0x3FFF`
12	`port`	U7	失敗したリクエストの `port`。この欄がないリクエストでは `0x7F`

DRAFT

off	フィールド名	型	意味
13	`error_code`	U7	失敗の理由

error_code	エラー名	意味
`0x01`	`BAD_REQUEST`	リクエストそのものが不正
`0x02`	`UNSUPPORTED_FUNCTION_ID`	未対応の機能
`0x03`	`BAD_PORT`	`port` が `0x00`–`0x7E` の範囲外、または指定したチャンネル・割り当て先が `CAPABILITY` にない
`0x04`	`NOT_PRESENT`	センサー未接続
`0x05`	`MEASURE_FAILED`	検出はできたが値が得られない
`0x06`	`ROLE_CONFLICT`	入力操作を指定した割り当て先が出力状態
`0x07`	`BUSY`	一時的に受け付けられない
`0x08`	`DEVICE_RESET`	起動後、デバイスがまだ `CAPABILITY` を受けていない
`0x09`	`UNSUPPORTED_COMMAND`	デバイスがそのコマンドを実装していない
`0x0A`	`CANCELLED`	受け付けた処理が、新しいリクエストや通信途絶によって取り消された

`READ`、`WRITE`、`INPUT_SUBSCRIBE` では、`CAPABILITY` に示された `(function_id, port)` を指定します。デバイスがその機能番号をどの `port` にも宣言していない場合は `UNSUPPORTED_FUNCTION_ID`、その機能番号は宣言済みでも指定した組がない場合は `BAD_PORT` を返します。入力機能への `WRITE` と出力機能への `READ` は `BAD_REQUEST` とし、割り当て先の状態は変えません。

`PORT_NAME` と `PORT_RELEASE` には、`CAPABILITY` に示された `port` を指定します。存在しない `port` には `BAD_PORT` を返します。`0x7F` は割り当て先がないことを示す値で、操作対象の `port` には指定できません。

デバイスは、構文が不正なリクエストに `BAD_REQUEST` を返します。`transaction_id` を読み取れないほど短いメッセージや、デバイス送信専用の `ACK` / `ERROR` / `PENDING` / `INPUT_EVENT` を受信した場合は、応答せずに破棄します。これにより、応答同士がループすることを防ぎます。それ以外の未対応コマンドは、ホストからのリクエストとして扱います。

まず 4.1 に従って SysEx の枠と `payload_length` を検証します。枠の長さ違反は、コマンドの対応有無より先に処理します。枠が正しいメッセージで複数のエラー条件が同時に成り立つ場合、デバイスは次の順で判定し、最初に該当したエラーを返します。

1. デバイス送信専用コマンドと、`transaction_id` を読み取れないほど短いメッセージは応答せずに破棄する
2. コマンド番号と `transaction_id` を読み取れる未対応コマンドには `UNSUPPORTED_COMMAND` を返す
3. 対応するコマンドの必須フィールドがないなど、メッセージの形式が不正な場合は `BAD_REQUEST` を返す

DRAFT

4. 起動後、`CAPABILITY` の `index = 0` に対する正常レスポンスを返す前の `READ` / `WRITE` / `PORT_RELEASE` / `INPUT_SUBSCRIBE` には `DEVICE_RESET` を返す
5. `READ` / `WRITE` / `INPUT_SUBSCRIBE` で未対応の `function_id` が指定された場合は `UNSUPPORTED_FUNCTION_ID` を返す
6. 対応する機能またはコマンドに指定された `port` がない場合は `BAD_PORT` を返す
7. 機能とコマンドの組み合わせ、値の形式または値域が不正な場合は `BAD_REQUEST` を返す
8. 入力状態・出力状態が合わない操作には `ROLE_CONFLICT` を返す
9. 一時的な処理上限に達している場合は `BUSY` を返す
10. センサーが接続されていない場合は `NOT_PRESENT` を返す
11. センサーから値を取得できない場合は `MEASURE_FAILED` を返す

`WRITE` の値形式と値域は、指定された `function_id` と `port` を確認してから判定します。`PORT_NAME` と `PORT_RELEASE` の `port` も、対応する割り当て先の有無を確認してから判定します。

- デバイスが `BUSY` を返す場面は、レスポンス待ちのリクエストが実装の上限に達したとき、アナログ・デジタル変換中に同じ割り当て先へ `READ` が届いたとき、など
- ホストは `BUSY` を受けたら再試行してよい
- デバイスは、再試行で解決する見込みの薄い失敗には `BUSY` 以外のコードを返す（箇条 9 の 14）
- 判定順の 10 と 11 は `READ` と `INPUT_SUBSCRIBE` の値取得時に適用する。通知登録後にセンサーの値を取得できなくなっても、デバイスは `INPUT_EVENT` を送らない。ホストが状態を確認する場合は `READ` を送る

5.13 `PENDING`（`0x09`）

`PENDING` は、処理が 100 ms 以内に終わらないリクエストに対し、デバイスが 100 ms 以内に返すメッセージです。処理中であることと、新しい期限を伝えます。その後、同じ `transaction_id` の正常レスポンスか `ERROR` が返ります。`PENDING` を返した時点でリクエストは受け付け済みです。その後に `ERROR` を返しても、受け付けた時点の状態変更は取り消しません。100 ms 以内に処理が終わる場合は、成功したときだけ状態を変えます。

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストの値をそのまま返す
9	`request_command`	U7	リクエストのコマンド番号
10-11	`function_id`	U14	リクエストの値をそのまま返す。この欄がないリクエストでは `0x3FFF`
12	`port`	U7	リクエストの値をそのまま返す。この欄がないリクエストでは `0x7F`
13-14	`wait`	U14	新しい期限までのミリ秒。`PENDING` を送った時点からの値

- `wait` は 0–16,383 ms の範囲
- デバイスは 1 つのリクエストに対して `PENDING` を複数回送ってよい。そのつど期限が延びる
- 期限は `PENDING` を送った時点から `wait` ミリ秒後とする
- デバイスは期限より早くレスポンスを返してよい

DRAFT

5.14 `READ`（`0x10`）

ホストは `READ` でチャンネルの値を 1 つ取得します。リクエストには、`CAPABILITY` が返した `(function_id, port)` を指定します。

リクエスト

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストごとに変える番号
9-10	`function_id`	U14	読み取る機能番号
11	`port`	U7	読み取り先

レスポンス

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストの値をそのまま返す
9-10	`function_id`	U14	リクエストの値をそのまま返す
11	`port`	U7	リクエストの値をそのまま返す
12-	値	—	測定値。データ長と形式は `function_id` ごとに機能表が定める

値のバイト数は `function_id` ごとに決まります。たとえば 3 成分を返す `ACCEL` の値は 12 byte です。

5.15 デバイス起点の状態変化通知

対象と目的

`READ` は、ホストが問い合わせた時点の値を返します。ポーリングの間に入力が変化して元に戻ると、その変化をホストが読めないことがあります。

ホストは入力チャンネルを登録すると、その値が変わったときに `INPUT_EVENT` を受け取れます。チャンネルを登録する操作を「通知登録」、届く `INPUT_EVENT` を「通知」と呼びます。通知登録できるのは、機能表で `kind: boolean` かつ `access: read` とされたチャンネルです。`DIGITAL_IN`、人感センサー、リミットスイッチなどに使えます。現在値は引き続き `READ` でも取得できます。通知機能は任意です。実装しないデバイスは `INPUT_SUBSCRIBE` と `INPUT_UNSUBSCRIBE` に `ERROR(UNSUPPORTED_COMMAND)` を返します。

入力値の確定

入力の走査方法には、一定間隔で読み取る方法や割り込みを使う方法などがあり、デバイスごとに異なります。スイッチのチャタリング処理を含め、入力値をどう確定するかもデバイス側で決めます。通知するのは、デバイスが確定した値の変化です。チャタリングのような一時的変化を別々のイベントとしてホストへ送る必要はありません。

通知の速さはデバイスによって異なります。入力の走査や値の確定処理、USB-MIDI の転送速度で決まり、この規格では最短間隔を定めません。ホストはメッセージだけから走査間隔やデバウンス時間、検出できる

DRAFT

最短パルス幅を知ることはできません。時間精度が必要な機器は、機器の説明書にこれらの条件と `device_time` の精度を記載します。

イベントの `device_time` には、入力値の変化を確定した時刻を設定します。これは物理的な変化が起きた時刻そのものや、検出遅延の上限を示すものではありません。USB-MIDI の転送やホストの処理による遅延を避けるため、パルス間隔や押下時間の計算にはホストの受信時刻ではなく、デバイス時刻を使います。短押し・長押しの判定方法自体は S-LINK では定めません。

入力の走査間隔や確定処理より短い変化は、デバイスが検出できないことがあります。この規格は、電気的な変化をすべて検出することまでは保証しません。

通知登録の開始と解除

1 台のデバイスを同時に操作するホストは 1 つです。デバイスは MIDI 接続の開閉を検出できないため、通知登録をデバイス上で管理します。ホストは通知登録ごとに、稼働中に再利用しない `subscription_id` を割り当てます。同じ `(function_id, port)` を再登録すると、古い登録は新しい登録に置き換わります。ホストは起動時に必要なチャンネルを登録し、現在使っている `subscription_id` と異なる通知は読み飛ばします。前回の登録 ID を保存している場合は、起動時に不要な登録を解除できます。デバイスが再起動するか、2 秒間リクエストを受け取らなければ、古い登録は解除されます。

コマンド名	用途
`INPUT_SUBSCRIBE`	指定した `(function_id, port)` の通知登録を開始する
`INPUT_UNSUBSCRIBE`	指定した `(function_id, port, subscription_id)` に一致する通知登録を解除する

通知に対応しないデバイスは、これらのリクエストに `ERROR(UNSUPPORTED_COMMAND)` を返します。ホストはその場合、従来どおり `READ` でポーリングします。`INPUT_SUBSCRIBE` の対象は、機能表に `kind: boolean` かつ `access: read` として掲載されたチャンネルです。未対応の `function_id` には `ERROR(UNSUPPORTED_FUNCTION_ID)` を返し、未対応の `(function_id, port)` には `ERROR(BAD_PORT)` を返します。機能と割り当て先は対応していても `kind` または `access` が対象外の場合は `ERROR(BAD_REQUEST)` を返します。

`INPUT_SUBSCRIBE` は `READ` と同様に割り当て先を入力状態にします。出力状態の割り当て先では `ERROR(ROLE_CONFLICT)` を返します。正常レスポンスを作る時点で、デバイスは現在値と `device_time` を取得し、イベント番号を 0 にします。これを通知登録時の基準とし、それ以降の変化には 1 から番号を付けます。基準値の取得と通知登録の開始は一続きの処理として行い、その間の変化を取りこぼしません。デバイスは、`INPUT_SUBSCRIBE` の応答を、その後の `INPUT_EVENT` より先に同じ MIDI 出力ポートから送ります。初期値を取得できない場合は `NOT_PRESENT` または `MEASURE_FAILED` を返し、通知登録を開始しません。登録後に値を取得できなくなった場合は、値の通知を行いません。ホストは必要に応じて `READ` で状態を確認します。

`INPUT_SUBSCRIBE` の処理中に、同じ割り当て先への `WRITE` または `PORT_RELEASE` を受け付けた場合や、通信が途絶えた場合は、7.1 と箇条 8 に従ってその処理を取り消します。取り消した処理が後から通知登録を開始することはありません。

`INPUT_SUBSCRIBE` を再度受けると、デバイスは通知登録の基準値を更新します。ホストは現在値とイベント番号を取り直せます。再登録には以前と異なる `subscription_id` を使い、イベント番号は 0 に戻します。応答より前に届く通知は旧登録のもの、応答の後に届く通知は新登録のものです。応答後、デバイスは旧登録の `subscription_id` を持つ通知を送りません。送信待ちの旧通知も破棄します。

`INPUT_UNSUBSCRIBE` は、`function_id`、`port`、`subscription_id` がすべて一致する通知登録を解除します。割り当て先の状態は変わりません。一致する登録がない場合も成功として扱います。デバイスは、それまでに発生したイベントを応答より先に送り、応答の後には送りません。同じチャンネルへの登録と解除は、受信順に処理します。通知登録中の割り当て先に `PORT_RELEASE` または出力への `WRITE` が届いた場合も登録を解除します。すでに発生したイベントは、`ACK` より先に送り、その後は送りません。

デバイスの再起動で通知登録は解除されます。ホストは接続手順をやり直し、通知が必要なチャンネルを再登録します。このときも以前と異なる `subscription_id` を使います。通知登録数の上限はデバイスが定め、上限を超える要求には `ERROR(BUSY)` を返します。ホストは、既存の通知登録を解除するなどして空きができてから再試行します。

通知登録と `KEEPALIVE` だけを使うホストは、デバイスの再起動を検出できません。再起動を検出する必要があるホストは、定期的に `READ` を送り、`ERROR(DEVICE_RESET)` を受けたら接続手順をやり直して通知登録を再開します。

メッセージ形式

通知登録を維持するホストは、出力中の端子がなくても、1 秒以内ごとに `KEEPALIVE` かほかのリクエストを送ります。デバイスは 2 秒間リクエストを受け取らないと通知登録を解除します。

`INPUT_SUBSCRIBE` と `INPUT_UNSUBSCRIBE` はリクエストへのレスポンスを返します。`INPUT_EVENT` はリクエストへの応答ではない、デバイス起点の通知です。`INPUT_EVENT` には `transaction_id` を含めず、ホストが指定する `subscription_id` とイベント番号で通知登録と通知を識別します。通知は、リクエストを送った MIDI 出力ポートに対応する MIDI 入力ポートへ送ります。

コマンド番号	コマンド名	方向	全長
`0x11`	`INPUT_SUBSCRIBE`	H→D / D→H	19 / 30 byte
`0x12`	`INPUT_UNSUBSCRIBE`	H→D / D→H	19 / 19 byte
`0x13`	`INPUT_EVENT`	D→H	28 byte

`INPUT_SUBSCRIBE`

リクエストのフィールドは次のとおりです。

off	フィールド名	型	意味
7–8	`transaction_id`	U14	リクエストごとに変える番号
9–10	`function_id`	U14	通知登録する入力機能
11	`port`	U7	通知登録する割り当て先
12–17	`subscription_id`	U42	ホストが通知登録に割り当てる識別番号

DRAFT

正常レスポンスには次のフィールドを含めます。

off	フィールド名	型	意味
7–8	`transaction_id`	U14	リクエストの値をそのまま返す
9–10	`function_id`	U14	リクエストの値をそのまま返す
11	`port`	U7	リクエストの値をそのまま返す
12–17	`subscription_id`	U42	リクエストの値をそのまま返す
18–21	`event_number`	U28	この通知登録のイベント番号。開始時は 0
22–27	`device_time`	U42	通知登録時のデバイス時刻
28	`value`	U7	通知登録時の現在値。機能表に定める値

`INPUT_UNSUBSCRIBE`

リクエストには `INPUT_SUBSCRIBE` と同じフィールドを含めます。正常レスポンスはリクエストと同じコマンド番号を使い、すべてのフィールドをそのまま返します。

off	フィールド名	型	意味
7–8	`transaction_id`	U14	リクエストの値をそのまま返す
9–10	`function_id`	U14	リクエストの値をそのまま返す
11	`port`	U7	リクエストの値をそのまま返す
12–17	`subscription_id`	U42	リクエストの値をそのまま返す

`INPUT_EVENT`

off	フィールド名	型	意味
7–12	`subscription_id`	U42	対応する通知登録の識別番号
13–16	`event_number`	U28	通知登録ごとに増えるイベント番号
17–18	`function_id`	U14	変化した入力機能
19	`port`	U7	変化した割り当て先
20–25	`device_time`	U42	値の変化を確定した時刻
26	`value`	U7	変化後の値。機能表に定める値

ホストは通知登録ごとに未使用の `subscription_id` を割り当てます。イベント番号は通知登録ごとに 0 から始め、イベントごとに 1 増やします。`268,435,455` の次は 0 に戻ります。デバイスは同じ通知登録のイベントをイベント番号順に送ります。ホストは現在の `subscription_id` と異なるイベントを読み飛ばし、`INPUT_SUBSCRIBE` の応答に含まれる番号を基準に次のイベント番号と比較します。番号が飛んでいれば通知の欠落を検出できます。ただし、欠落数が `2^28` の倍数の場合は検出できません。再起動後は通知登録が解除されるため、ホストは以前と異なる `subscription_id` を割り当てて再登録します。

デバイス時刻は起動後の単調増加時間で、マイクロ秒単位とします。`U42` の値は約50.9日で周回します。後の `device_time` から前の値を引き、差が負になった場合は `2^42` を足します。この方法で実際の経過時間

を得られるのは、2 つの時刻の間隔が約50.9日未満の場合です。

```
前の時刻  2^42 − 1000
次の時刻  500
差        500 − (2^42 − 1000) + 2^42 = 1500 µs
```

時刻は機器間の同期には使いません。実際の測定精度はデバイスの時計精度に依存します。

通知の欠落

デバイスは、通知に対する確認応答を待たず、通知を再送しません。

- イベント番号が飛んだら、通知の欠落です。後続イベントから現在値は分かりますが、欠落したイベントは復元できません。最後のイベントを取りこぼし、その後に通知がなければ、欠落を検出できません。
- 欠落を検出したホストは `INPUT_SUBSCRIBE` を再送し、応答の現在値とイベント番号を新しい基準にします。古い `subscription_id` のイベントは読み飛ばします。`READ` は値を返しますが、イベント番号の基準を更新しません。

5.16 `WRITE`（`0x20`）

`WRITE` はデバイスの出力を制御するコマンドです。サーボモーターを動かしたり、LED を点灯させたりするときに使います。

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストごとに変える番号
9-10	`function_id`	U14	駆動する機能番号
11	`port`	U7	駆動する対象
12-	値	—	駆動する量。データ長と形式は `function_id` ごとに機能表が定める

- `WRITE` も `READ` と同じく `(function_id, port)` で指定する。出力機器の種類ごとにコマンドを分けない
- 値の形式は、機能表の `payload` が定める

以下は機能表に定める値の例です。`LEVEL` は on/off と可変の両方を表します。デジタルの 1 は 100 %、0 は 0 % です。 0 と 100 % だけを受け付ける `LEVEL` の割り当て先では、デバイスが `flags` bit2 を 1 にします。 その割り当て先へ 0 と 100 % 以外の値が届いたとき、デバイスは `ERROR(BAD_REQUEST)` を返します。機能表の説明欄が機能の有効範囲を定めている場合、その範囲を外れた `WRITE` 値にも `ERROR(BAD_REQUEST)` を返します。指数・値域・分解能の例示欄だけを根拠に値を拒否してはなりません。値を丸めたり範囲内に収めたりしません。

`RGB` の各成分を 7 bit で扱う場合の例を示します。ホストは 0–255 の色から次のように作ります。

```
r7 = r8 >> 1      255 → 127、128 → 64、1 → 0
```

DRAFT

デバイスは、同じ割り当て先に複数の出力機能（たとえば `LEVEL` と `SERVO`）を宣言できます。 割り当て先の出力は、最後に受理した `WRITE` の機能と値に従います。`LEVEL` に 50 % を書いた後に `SERVO` に角度を書くと、前の出力を置き換えてサーボモーターをその角度へ動かします。 どちらの機能に値を書いても、割り当て先は出力状態になります。

5.17 `PORT_RELEASE`（`0x21`）

`PORT_RELEASE` は指定した割り当て先の駆動を停止し、解放状態に戻すコマンドです。物理端子は高インピーダンス（Hi-Z）になります。出力に使っていた対象を入力に戻す場合にも使います。

off	フィールド名	型	意味
7-8	`transaction_id`	U14	リクエストごとに変える番号
9	`port`	U7	解放する割り当て先

`ENCODER` や `ENERGY` のように累積値を返す入力チャンネルは、`PORT_RELEASE` ではリセットされません。値が 0 に戻るのは電源投入時だけです。

サーボモーターは駆動を続けることで位置を保持します。駆動を止めるときは `PORT_RELEASE` を送ります。

5.18 レスポンスの対応

デバイスは、構文を解釈できたリクエストに正常レスポンスまたは `ERROR` を返します。`transaction_id` を読み取れないメッセージと、デバイス宛ではないコマンドは応答せずに破棄します。`INPUT_EVENT` はリクエストに対する応答ではなく、通知登録中の値が変化したときにデバイスが送る通知です。

```
HELLO / MODEL_ID / DEVICE_ID        →  同じコマンド番号のレスポンス  または  ERROR
FIRMWARE_VERSION                    →  同じコマンド番号のレスポンス  または  ERROR
CAPABILITY / PORT_NAME / KEEPALIVE  →  同じコマンド番号のレスポンス  または  ERROR
READ                                →  同じコマンド番号のレスポンス  または  ERROR
INPUT_SUBSCRIBE / INPUT_UNSUBSCRIBE →  同じコマンド番号のレスポンス  または  ERROR
WRITE / PORT_RELEASE                →  ACK                          または  ERROR
INPUT_EVENT                         →  通知登録中の入力値の変化を通知
```

---

6 機能表

機能表では、各機能の番号、単位、データ形式、動作を定めます。機能表は規格の発行者が管理します。

機能表には全体で 1 つの版番号を付けます。デバイスは `HELLO` のレスポンスで、準拠する機能表の版を `function_table_revision` として通知します。ホストが知らない `function_id` を受信した場合、その値は読み飛ばします。

- 機能表には、入力機能（センサーなど）と出力機能（サーボモーター、リレー、LED など）の両方を掲載する

- 機能番号とコマンド番号は、別の番号空間
- `function_id` によって値の意味が決まる。1 つの `function_id` に対応するデータ形式は 1 種類とする

6.1 値の種別

`kind`	意味	例
`adc`	A/D 値。`bits` と `reference_mv` で電圧に直せる	`ANALOG_IN`
`quantity`	物理量。単位と指数を定める	`ILLUM`（lx）／`TEMP`（degC）
`boolean`	2 値。各値の意味を定める	`PIR`（0 = 検知なし、1 = 検知あり）
`color`	色。成分ごとに値を持つ	`RGB`（R / G / B 各 0–127）

換算前の A/D 値が必要な場合、ホストは `kind: adc` の `ANALOG_IN` を読み取ります。

6.2 単位と値の換算

単位はメッセージに含まれません。ホストは `function_id` に対応する単位を機能表で確認し、`CAPABILITY` が示す `exponent` を使って受信値を物理量に換算します。

```
換算例            function_id 0x0300   name ILLUM   kind quantity   unit lx
CAPABILITY        そのチャンネルの exponent = −2

送受信する値
  50000

ホストの計算
  50000 × 10⁻² = 500.00
  画面に出す: 500 lx
```

`payload` に複数の欄がある場合、`exponent` は機能表の `単位` 列に対応する欄に適用します。3 軸の値には、3 成分すべてに同じ倍率を適用します。

```
ACCEL   payload  x / y / z S28   単位 m/s2   exponent −4

  x   10000  ×10⁻⁴  →   1 m/s²
  y  −20000  ×10⁻⁴  →  −2 m/s²
  z   30000  ×10⁻⁴  →   3 m/s²
```

単位が異なる欄には `exponent` を適用しません。

```
GATE_TIME   payload  duration U28 / count U14   単位 s   exponent −6

  duration  20000  ×10⁻⁶  →  0.02 s
  count         5  そのまま  →  5 回
```

出力値を送るときは、指定する物理量を 10^exponent で割ってメッセージの値に換算します。

```
機能表            function_id 0x1001   name SERVO   kind quantity   unit deg
CAPABILITY        そのチャンネルの exponent = −1

サーボモーターを 90 度に向ける
  90 ÷ 10⁻¹ = 900

送受信する値
  900
```

6.3 機能表の項目

列	意味	区分
`function_id`	機能番号。範囲ごとの用途は、機能表の「機能番号の範囲」で定める	規定
`name`	識別子。規格書とコードで使う	規定
表示名 / English	ホストが画面に表示する名前	規定
`access`	`read`（`READ` で値を得る）か `write`（`WRITE` で駆動する）	規定
`kind`	値の種別。`adc` / `quantity` / `boolean` / `color`	規定
単位	`kind: quantity` のときの単位。指数は別に指定する	規定
`payload`	送受信するフィールドの並び	規定
説明	測定値の意味や出力の動作	規定
指数（例） / 値域 / 分解能	参考となる値	例

`kind: boolean` の値 0 と 1 が何を表すかは、機能表の「`boolean` の値の意味」の表で確認できます。

機能表の `payload` が定める値は 61 byte 以下とします。`READ` のレスポンスと `WRITE` のペイロードには、値のほかに `transaction_id`、`function_id`、`port` の計 5 byte が入るためです。これを超える形式は、この規格のメッセージでは扱えません。`WRITE` の値の長さが機能表の定義と異なる場合は `ERROR(BAD_REQUEST)` を返します。

`payload` 列の読み方、フィールドの順序、型と指数の適用方法は、この節で定めます。機能表にはその機能で使うフィールドを示します。

```
value U28                    value（U28、4 byte）              計 4 byte
duration U28 / count U14     duration（U28）→ count（U14）      計 6 byte
x / y / z S28                x → y → z（それぞれ S28）          計 12 byte
index U14 / r / g / b U7     index（U14）→ r → g → b（U7）      計 5 byte
```

- `payload` によって値の長さが決まる。`READ` のレスポンスと `WRITE` の値の形式は、この並びから一意に決まる
- `exponent` はチャンネルごとに `CAPABILITY` で宣言する。ホストはその値を使って送受信値を解釈する

DRAFT

- `exponent` はチャンネルごとに 1 つだけ指定できる。機能表の `単位` 列と同じ単位を持つすべての欄に適用する
- それ以外の欄は `exponent` を適用せず、その単位の整数として扱う。各欄の単位は機能表に記載する
- `kind: quantity` 以外のチャンネルでは、`exponent` を 0 とする
- 機能表に記載された単位と `boolean` の値の意味は、ホストが使用する
- チャンネルディスクリプターの `flags` bit0 は、該当する `function_id` の `access` と一致させる。両者が異なる場合は機能表の `access` を正とする
- 規格の発行者は、機能を追加するときに機能表の版番号を 1 増やして新しい行を追加し、既存の行は変更しない

6.4 非公式の機能番号

`0x3E00`–`0x3EFF` は、機器の開発者が独自の機能に使える 256 個の番号です。同じ番号を別の機器が使ってもかまいません。番号ごとの機能の意味は、その機器の開発者が決めます。 これらの番号も `CAPABILITY` で宣言し、通常の `READ` / `WRITE` で値をやり取りします。機器の開発者は、番号ごとの意味、入出力の別、単位、値の型・並び・長さ・範囲、指数の適用方法を説明書に記載します。値は 4.3 のデータ型で表し、符号化後 61 byte 以下とします。`payload_length` には共通フィールドの 5 byte も含めます。応答、エラー、入出力の状態、`PORT_RELEASE`、通信途絶時の動作も既存の規定に従います。

この番号帯は機能表に載らないため、他社製のホストは通常、番号の意味を解釈できません。ホストは知らない機能番号を読み飛ばします（箇条 6）。この番号帯の機能は、S-LINK で相互運用できる機能として扱いません。

複数の機器の開発者が同じ番号を使うことがあります。たとえば、A 社と B 社の `0x3E00` は別の機能を表す場合があります。 ホストは対象の接続で得た `model_id` で機種を確認してから、この番号帯を解釈します（箇条 9 の 15）。

機能表に正式な番号がある機能でも、デバイスは旧番号と正式番号の両方を `CAPABILITY` で宣言できます。例えば、明るさを表す機能を非公式番号から `ILLUM` に移す間は、次のように両方を宣言できます。

```
(0x3E00 非公式の明るさ, port 2)    古いホストが読む
(0x0300 ILLUM,         port 2)     新しいホストが読む
```

同じ割り当て先に複数の入力 `function_id` を割り当てられるため、ファームウェアを更新しても旧ホストとの互換性を保ちながら正式な番号へ移行できます。

---

7 割り当て先の状態

入力機能を持つチャンネルを「入力チャンネル」、出力機能を持つチャンネルを「出力チャンネル」と呼びます。ホストは入力チャンネルを `READ` し、出力チャンネルに `WRITE` します。

デバイスは `port` ごとに、入力・出力・解放の状態を管理します。この区別は通信上のもので、対象が物理端子か機器内の機能かは問いません。

- DS18B20（1-Wire）のように 1 本の線で送受信するセンサーも、規格の上では入力チャンネルである。向きの反転はファームウェアの中で完結し、メッセージには現れない
- 同じ割り当て先に複数の入力機能を宣言してよい（`ANALOG_IN` と `ILLUM` など）。ただし、入力機能ごとに測定回路や端子設定が異なることがあり、一方の `READ` や通知登録が他方の測定・通知に影響する場合がある。機器の説明書に、同じ割り当て先にある入力機能の併用条件を記載する

同じ割り当て先を入力と出力に同時に使うことはできません。たとえば、`port 0x00` でサーボモーターを駆動している間に、その割り当て先のスイッチを読むリクエストを送ると、デバイスは `ERROR(ROLE_CONFLICT)` を返します。

同じ割り当て先を入力と出力に切り替えて使うことはできます。サーボモーターの駆動後に `PORT_RELEASE` を送り、割り当て先を解放してからスイッチを読みます。

7.1 入力・出力の切り替え

デバイスは、受け取ったコマンドに応じて割り当て先の状態を切り替えます。

```
解放状態（物理端子は高インピーダンス）　　初期状態
  READ            → 入力状態へ遷移
  INPUT_SUBSCRIBE → 入力状態へ遷移し、通知を開始
  INPUT_UNSUBSCRIBE → 通知なしとして成功。状態はそのまま
  WRITE           → 出力状態へ遷移
  PORT_RELEASE    → そのまま

入力状態
  READ            → 値を返す
  INPUT_SUBSCRIBE → 通知を開始または再開始
  INPUT_UNSUBSCRIBE → 通知を終了。入力状態のまま
  WRITE           → 出力状態へ遷移。処理中の READ / INPUT_SUBSCRIBE には ERROR(CANCELLED) を返す
  PORT_RELEASE    → 解放状態へ遷移

出力状態
  READ            → ERROR(ROLE_CONFLICT) を返す。出力状態のまま
  INPUT_SUBSCRIBE → ERROR(ROLE_CONFLICT) を返す
  INPUT_UNSUBSCRIBE → 通知なしとして成功。出力状態のまま
  WRITE           → 受理する
  PORT_RELEASE    → 解放状態へ遷移
```

デバイスは、電源投入後にすべての割り当て先を解放状態とします。 `PORT_RELEASE` を受けた割り当て先も解放状態に戻します。

`CAPABILITY` は対応するチャンネルを示すだけで、割り当て先の状態は変えません。100 ms を超える処理では、デバイスが `PENDING` を返した時点でリクエストを受け付け、割り当て先の状態を切り替えます。そ

の後に `ERROR` を返しても状態は元に戻しません。100 ms 以内に処理が終わる場合は、成功時に状態を切り替え、`ERROR` のときは変えません。

入力状態から出力状態へは `WRITE` で切り替えます。出力状態から入力に切り替えるときは、`PORT_RELEASE` で解放してから `READ` を送ります。 出力の駆動を停止する操作には `PORT_RELEASE` を使用します。

入力状態から出力状態へ切り替える `WRITE` を受け付けた場合、最終レスポンスをまだ返していない同じ割り当て先への `READ` / `INPUT_SUBSCRIBE` は終了し、それぞれの `transaction_id` に `ERROR(CANCELLED)` を返します。拒否した `WRITE` では、先の処理を取り消しません。すでに最終レスポンスを返した処理も取り消しません。デバイスは、取消しエラーを `WRITE` の `ACK` または `PENDING` より先に送ります。出力状態の割り当て先に新しい `READ` または `INPUT_SUBSCRIBE` を送った場合は、`ERROR(ROLE_CONFLICT)` を返します。

`PORT_RELEASE` を受け付けた場合も、同じ割り当て先を対象とする処理中の `READ` / `INPUT_SUBSCRIBE` / `WRITE` を取り消し、それぞれに `ERROR(CANCELLED)` を返します。デバイスは、これらのエラーを `PORT_RELEASE` の `ACK` または `PENDING` より先に送ります。この規則は、すでに `PENDING` を返した処理にも適用します。

処理中の `WRITE` がある割り当て先に新しい `WRITE` が届き、それを受け付ける場合、デバイスは先の処理を取り消して `ERROR(CANCELLED)` を返してから、新しい `WRITE` を処理します。先の処理の取消しエラーは、新しい `WRITE` の `ACK` または `PENDING` より先に送ります。すでに最終レスポンスを返した処理は取り消しません。

処理を取り消した場合、それ以降の駆動は止めます。すでに生じた動作や物理的な影響は取り消せません。

入力チャンネルを通知登録中に、その割り当て先への `WRITE` または `PORT_RELEASE` を受け付けた場合、デバイスはその通知登録を解除します。拒否したリクエストでは解除しません。すでに生成したイベントは、`WRITE` または `PORT_RELEASE` の `ACK` より先に送り、ACK の後はイベントを送りません。`INPUT_UNSUBSCRIBE` も同様に、生成済みイベントを正常レスポンスより先に送ります。

`READ` や `WRITE` を送っても、`CAPABILITY` の内容は変わりません。`CAPABILITY` を読み直す必要があるのは、`ERROR(DEVICE_RESET)` を受けたときです。

```
WRITE(SERVO, port 0)       →  ACK    割り当て先は出力状態
PORT_RELEASE(port 0)       →  ACK    割り当て先は解放状態
READ(DIGITAL_IN, port 0)   →  値      割り当て先は入力状態
```

同じ割り当て先が複数の入力機能に対応している場合、ホストは `READ` のたびに 1 つを選びます。 次の例では、どちらを読んでも割り当て先は入力状態のままです。

```
READ(ANALOG_IN, port 2)   →  2048        A/D 値
READ(ILLUM,     port 2)   →  50000       ホストが 500 lx と表示する
```

DRAFT

7.2 物理端子の電気的性質（`pull_mv` / `pull_ohm`）

端子にプルアップ抵抗・プルダウン抵抗・分圧回路などが接続されていると、読み取り値は外部から加わる電圧と回路の影響を受けます。 `flags` の bit1 が 1 のとき、`pull_mv` と `pull_ohm` は、そのチャンネルを使うときに外部負荷を接続しない状態で端子から見える回路を表します。これは開放時電圧とテブナン等価抵抗であり、回路内にその値の抵抗が 1 個あることを意味しません。

```
470 kΩ ずつで 5V を分圧    pull_mv = 2500    pull_ohm = 235000
3.3 V 電源へ 10 kΩ              pull_mv = 3300    pull_ohm =  10000
接地（GND）へ 10 kΩ              pull_mv =     0    pull_ohm =  10000
何も入っていない              flags bit1 = 0
```

`flags` の bit1 が 0 のとき、デバイスは 2 つの欄に 0 を入れ、ホストは 2 つの欄を読み飛ばします。

`pull_mv` と `pull_ohm` はチャンネルごとに設定します。同じ物理端子でも、チャンネルごとに有効になる抵抗回路が異なれば値も異なります。

端子と GND の間に抵抗をつなぎ、`pull_mv > 0` かつ `0 < V < pull_mv` の場合、ホストは次の式でその抵抗を求めます。`pull_mv = 0` の場合、この式は使えません。

```
読み取り値 V[mV] のとき    R = pull_ohm × V ÷ (pull_mv − V)
                          G = (pull_mv − V) ÷ (pull_ohm × V)
```

`G`（コンダクタンス）の式なら、端子が開放状態（`V = pull_mv`）のときに 0 になります。 同じ条件では `R` の式が 0 除算になるため、開放状態の判定には `G` を使います。

7.3 外付けのアナログセンサー

アナログ端子が測定するのは電圧です。デバイスは、接続されたセンサーの種類までは判別できません。 ホストは `CAPABILITY` に含まれる機能を見て、受信値の意味を判断します。

デバイスの宣言	`READ` が返す値
`ANALOG_IN`	A/D 値。ホストが解釈する
`ILLUM` など具体的な機能	換算済みの値。デバイスが換算する

ホストは `bits`、`reference_mv`、`pull_mv`、`pull_ohm` を使って `ANALOG_IN` の値を解釈します。 分圧回路を備えた端子では、同じセンサーを接続しても機器によって A/D 値が異なります。

---

8 安全と個体識別

USB のサスペンド、リセット、デタッチをデバイスが検出した場合の動作は、推奨事項 28（箇条 9）で定めます。

例えば、ホストのタブを閉じたりアプリケーションを終了したりしても、USB 接続は切れません。終了したこともデバイスには通知されません。 このため、デバイスはリクエストが途絶えたことからホストの停止を判断します。

デバイスは、形式の整った S-LINK リクエストを受信するたびに通信途絶の監視時間を更新します。値域違反など、処理の結果 `ERROR(BAD_REQUEST)` を返す場合も更新します。形式が不正なメッセージや S-LINK 以外のメッセージでは更新しません。`PENDING` などのレスポンスを送っても、この時間は更新しません。ホストから最後にリクエストを受信してから 2 秒が過ぎたら、出力状態の割り当て先を解放します。

出力中の端子や通知登録を維持するホストは、前回のリクエスト送信から 1 秒以内に次のリクエストを送ります。通知イベントはホストからのリクエストではないため、途絶監視時間を更新しません。500 ms 間隔なら、通信の遅れに余裕を持てます。

ホストは、出力状態の割り当て先がある間、または通知登録を維持する間、リクエストを定期的に送ります。定期的な `READ` があれば、それを動作確認に使えます。 ほかに送るリクエストがない場合は、`KEEPALIVE`（5.10）を送信します。

通信が 2 秒途絶えた場合、デバイスはすべての通知登録を解除し、送信待ちの通知も破棄します。通信途絶によって解放された割り当て先は、`PORT_RELEASE` を受けた場合と同じ状態になります。処理中の `READ` / `INPUT_SUBSCRIBE` / `WRITE` がある場合は取り消し、`ERROR(CANCELLED)` を返します。`PORT_RELEASE` の `ACK` は送信しません。

`PENDING` はホストからのリクエストではないため、通信途絶の監視時間を更新しません。1 秒を超える処理を待つホストも、出力を維持する間は `KEEPALIVE` または別のリクエストを 1 秒以内に送ります。ホストが復帰した後、`WRITE` を再送すれば出力を再開できます。

出力状態の割り当て先がなければ、リクエストが途絶えても割り当て先の状態は変わりません。通信途絶時に出力状態の割り当て先を解放する動作は、入力機能だけを持つ機器には適用しません。通知登録の解除と処理中リクエストの取り消しは、該当する機能や処理があれば入力機能だけの機器にも適用します。

機器の個体 ID をホストが取得する場合は、任意コマンド `DEVICE_ID` を使います。機種固有の追加情報を表す機能には、6.4 の非公式番号帯を使えます。機器の説明書では、`model_id` が機種、`device_id` が個体を示すことを説明します。

---

9 要求事項と推奨事項

この規格では、本文と箇条 9 の表で動作を定めます。「望ましい」と書いた動作は推奨事項で、省略しても通信できます。箇条 9 は本文の規定を探しやすくするための一覧です。表の件数は、箇条 9 に掲載した項目の数です。本文にある細かな動作規定も適用されます。例として示した値や処理は、明示がない限り要求事項ではありません。

要求事項には「します」「とします」、推奨事項には「望ましい」と書きます。任意機能は実装を省けることを本文に記します。

DRAFT

9.1 接続

#	対象	内容	区分
1	両方	未知のプロトコル識別子を受信したら、応答せずに破棄する	要求事項
2	デバイス	起動後、`CAPABILITY` の `index = 0` に対する正常レスポンスを返すまでは、`READ` / `WRITE` / `PORT_RELEASE` / `INPUT_SUBSCRIBE` に `ERROR(DEVICE_RESET)` を返す	要求事項
3	ホスト	ポート名にかかわらず、開いた各 MIDI 出力ポートへ探索全体で異なる `transaction_id` の `HELLO` を送り、入力ポートとの組を対応付ける（5.4）	要求事項
4	デバイス	`total > 0` では、`index < total` なら 5.8 の条件に従ってチャンネルディスクリプターを 1 個以上返し、`index` が `total` 以上なら `ERROR(BAD_REQUEST)` を返す。`total = 0` では `index = 0` に限り、チャンネルディスクリプター 0 個のレスポンスを返す	要求事項

9.2 リクエストとレスポンス

#	対象	内容	区分
5	デバイス	レスポンスにはリクエストの `transaction_id` をそのまま入れる	要求事項
6	ホスト	新たなリクエストには、同じ接続内でレスポンス待ちのリクエストとは異なる `transaction_id` を用いる。探索中は 5.4 に従う	要求事項
7	ホスト	`transaction_id` は前回使った値の次から探し、応答待ちまたは再利用待ちの番号を飛ばして割り当てることが望ましい	推奨事項
8	デバイス	形式を解釈できる S-LINK リクエストには、正常レスポンスか `ERROR` のどちらかを返す	要求事項
9	デバイス	`WRITE` と `PORT_RELEASE` には、成功時に `ACK` を、失敗時に `ERROR` を返す	要求事項
10	デバイス	未対応の `function_id` には `ERROR(UNSUPPORTED_FUNCTION_ID)` を返す	要求事項
11	両方	ペイロードは 66 byte 以内とし、`payload_length` に符号化後の長さを設定する。System Real Time を除いて長さを検証し、不一致・上限超過は 4.1 に従う	要求事項
12	デバイス	センサーの未接続と、測定の失敗を見分けて、それぞれのコードで返すことが望ましい	推奨事項
13	デバイス	実装していないコマンドには `ERROR(UNSUPPORTED_COMMAND)` を返す	要求事項
14	デバイス	再試行で解決する見込みの薄い失敗には、`BUSY` 以外のコードを返すことが望ましい	推奨事項
15	ホスト	`0x3E00`–`0x3EFF` の `function_id` は、`model_id` を確かめたうえで解釈する	要求事項
16	デバイス	非公式番号帯 `0x3E00`–`0x3EFF` を使う場合、ホストが機種別の扱いを選べるよう `MODEL_ID` を実装することが望ましい	推奨事項

DRAFT

#	対象	内容	区分
17	ホスト	同じデバイスに対して同時にリクエストを送るホストは 1 つとする。1 ホストから複数デバイスを使う場合は、1.2 に従い接続ごとに通信を管理する	要求事項
18	ホスト	各 MIDI 入力ポートで `HELLO` の応答を 1 秒待ち、応答がなければ 1 秒以上の間隔を空けて最大 2 回再送することが望ましい	推奨事項

9.3 割り当て先の状態

#	対象	内容	区分
19	デバイス	同じ割り当て先を入力と出力の両方に使う状態を作らず、`ERROR(ROLE_CONFLICT)` で退ける	要求事項
20	デバイス	割り当て先の状態は、受け取ったコマンドに応じて切り替える。状態を設定する専用コマンドは設けない	要求事項
21	デバイス	同じ `port` 値を持つチャンネルに、同じ割り当て先を割り当てる	要求事項
22	デバイス	出力状態の割り当て先への `READ` は `ERROR(ROLE_CONFLICT)` を返し、状態は変えない	要求事項
23	デバイス	`WRITE` で入力状態から出力状態へ切り替えるとき、処理中の `READ` / `INPUT_SUBSCRIBE` を終了して `ERROR(CANCELLED)` を返す。拒否した `WRITE` では先の処理を取り消さない	要求事項
24	デバイス	電源投入後、すべての割り当て先を解放状態とする。`PORT_RELEASE` を受けた割り当て先も解放状態に戻す	要求事項
25	機器の開発者	基板に端子名を印字する場合は、`PORT_NAME` が返す名前と合わせることが望ましい	推奨事項

9.4 MIDI

#	対象	内容	区分
26	両方	S-LINK 以外の MIDI メッセージを S-LINK として処理しない。他社宛の SysEx は 4.1 の受信処理に従って読み飛ばす。System Real Time は切り分け、S-LINK の長さに含めない	要求事項
27	両方	各デバイスの USB-MIDI 接続で Cable Number は 0 を用いる。別のデバイスの Cable Number 0 と混同しない	要求事項

9.5 安全

#	対象	内容	区分
28	デバイス	USB のサスペンド、リセット、またはデタッチを検出した場合、出力状態の割り当て先を解放することが望ましい	推奨事項
29	デバイス	形式の整った S-LINK リクエストを 2 秒受信しなければ、出力状態の割り当て先を解放し、通知登録を解除する。レスポンスでは監視時間を更新しない	要求事項

DRAFT

#	対象	内容	区分
30	ホスト	出力中の端子または通知登録を維持する間は、1 秒以内ごとに `KEEPALIVE` かほかのリクエストを送る	要求事項

9.6 時間

#	対象	内容	区分
31	デバイス	リクエストを受けてから 100 ms 以内に、正常レスポンス・`ERROR`・`PENDING` のいずれかを返す。`PENDING` を返した場合は、その送信時点から `wait` ミリ秒後を新しい期限とする	要求事項
32	ホスト	リクエストの期限（`PENDING` を受けた場合は、最後に `PENDING` が示した期限）を過ぎても応答がない場合、そのリクエストを失敗として扱い、その期限から 1 秒以上経ってから `transaction_id` を再利用することが望ましい	推奨事項

9.7 状態変化通知

#	対象	内容	区分
33	デバイス	状態変化通知を実装しない場合、`INPUT_SUBSCRIBE` / `INPUT_UNSUBSCRIBE` に `ERROR(UNSUPPORTED_COMMAND)` を返す。実装する場合は、5.15 に従う	要求事項
34	デバイス	`INPUT_SUBSCRIBE` に成功したら、通知開始時の値・時刻・イベント番号を返し、その後の変化を `INPUT_EVENT` で通知する	要求事項
35	デバイス	通知を解除するとき、生成済みイベントを `INPUT_UNSUBSCRIBE` の正常レスポンス、または `WRITE` / `PORT_RELEASE` の `ACK` より先に送る。再登録時と通信途絶時の送信待ち通知の扱いは、5.15 と箇条 8 に従う	要求事項
36	両方	ホストは通知登録ごとに異なる `subscription_id` を使い、`DEVICE_RESET` を検出したら接続手順をやり直して再登録する。デバイスは再起動時に通知登録を解除する	要求事項
37	ホスト	イベント番号の欠落を検出したら再登録し、応答の現在値と番号を新しい基準にする	要求事項

```
要求事項    29
推奨事項     8
対象        デバイス 22 / ホスト 9 / 両方 5 / 機器の開発者 1
```

---

10 最小構成

次のコマンドを実装すれば、S-LINK 2.0 で通信できます。デバイスは、対応しているチャンネルだけを宣言します。

コマンド名	用途	実装
`HELLO`	ホストが、S-LINK 2.0 デバイスの存在を確かめるために送る	必須

コマンド名	用途	実装
`CAPABILITY`	ホストが対応チャンネルの一覧を取得するために送る	必須
`ERROR`	デバイスが、リクエストが失敗した理由をホストへ伝えるために送る	必須
`READ`	ホストが、チャンネルの値を得るために送る	入力チャンネルを宣言する機器
`WRITE` `PORT_RELEASE`	ホストが、出力を駆動し、割り当て先を解放するために送る	出力チャンネルを宣言する機器
`ACK`	デバイスが、`WRITE` と `PORT_RELEASE` の成功をホストへ伝えるために送る	出力チャンネルを宣言する機器
`MODEL_ID`	ホストが、デバイスの機種を知るために送る	任意
`DEVICE_ID`	ホストが、デバイスの個体 ID を知るために送る	任意
`FIRMWARE_VERSION`	ホストが、ファームウェアの版を知るために送る	任意
`PORT_NAME`	ホストが、割り当て先の名前を知るために送る	任意
`KEEPALIVE`	ホストが、動作中であることを伝えるために送る	出力チャンネルまたは状態変化通知を実装する機器
`PENDING`	デバイスが、100 ms を超える処理で期限を延ばすために送る	任意
`INPUT_SUBSCRIBE` `INPUT_UNSUBSCRIBE` `INPUT_EVENT`	ホストが通知登録し、デバイスが値の変化を通知する	任意

`MODEL_ID`、`DEVICE_ID`、`FIRMWARE_VERSION`、`PORT_NAME` のいずれかを実装しないデバイスは、該当するリクエストに `ERROR(UNSUPPORTED_COMMAND)` を返します。これによりホストは、機種情報、個体 ID、ファームウェアの版、端子名のうち、どれを取得できないかが分かります。

`MODEL_ID`、`DEVICE_ID`、`FIRMWARE_VERSION`、`PORT_NAME` がなくても、値の読み書きはできます。チャンネル一覧は `CAPABILITY` から、単位と表示名は機能表から取得できるためです。

```
PORT_NAME がある     「デジタル入力 1」の明るさ    523 lx
PORT_NAME が無い     「port 0」の明るさ        523 lx
```

`PENDING` はデバイスからホストへ送るメッセージです。すべてのリクエストに 100 ms 以内で応答できるデバイスは、これを実装する必要はありません。

チャンネルがないデバイスも、最小構成を満たせます。その場合も `HELLO` に応答し、`CAPABILITY` の `total` に `0` を返します。

附属書 A（参考）接続から最初の値を読むまで

探索では候補の各 MIDI 出力ポートへ個別に送信します。手順 3 以降は、`HELLO` で確定した MIDI 入出力ポートの組ごとに行います。別の組は並行して扱えます。

```
1  MIDI ポートを開く
       すべてのポートに対して行う。ポート名で絞り込まない

2  出力ポートごとに異なる transaction_id で HELLO を送る
       1 秒待って応答を確認する。応答がなければ、1 秒以上の間隔を空けて最大 2 回再送することが望ましい
       レスポンスあり  → 受信元と transaction_id から MIDI 入出力ポートの組を確定し、function_table_revision を得る
       3 回とも応答なし → 対象外。ポートを閉じる

3  MODEL_ID を送る
       model_id を得る。画面表示を機種に合わせるために使う
       省略してよい。省略した場合、ホストは機種名を表示できず、機種固有の処理や非公式 function_id の意味を判別できない

4  CAPABILITY を送る
       (function_id, port) の一覧を得る
       1 回で floor((66 − 5) / descriptor_bytes) チャンネルまで。total に達するまで、実際に返された個数だけ index を進めて繰り返す

5  PORT_NAME を送る
       4 で出てきた port の値ごとに 1 回。割り当て先の名前を得る
       省略してよい。省略すると割り当て先は port 番号のまま表示される

6  READ を送る
       CAPABILITY に載っていた (function_id, port) を指定して値を読む
       通知を使う場合は INPUT_SUBSCRIBE を送り、応答の現在値を受け取る
```

- 抜き差し後も同じ個体を識別する場合は、接続時に `DEVICE_ID` を送り、得られた `device_id` を保存する。`ERROR(UNSUPPORTED_COMMAND)` が返っても、接続手順を続ける
- ファームウェアの版を表示・記録する場合は、接続時に `FIRMWARE_VERSION` を送る。`ERROR(UNSUPPORTED_COMMAND)` が返っても、接続手順を続ける
- 3 から 5 は接続時に 1 回でよい。以後は 6 を繰り返す
- `ERROR(DEVICE_RESET)` が返ったら、デバイスが再起動している。4 からやり直し、通知を使うチャンネルを再登録する

附属書 B（参考）ブラウザーでの複数台接続

Web MIDI API に対応し、SysEx の利用が許可されたブラウザーでは、1 つのアプリケーションから複数の MIDI 入出力ポートを利用できます。

ホストは `requestMIDIAccess({sysex: true})` で利用を開始し、`inputs` と `outputs` を列挙します。各出力ポートから `HELLO` を送信し、応答の `transaction_id` と受信元の入力ポートから、入出力ポートの組を対応付けます。

機器の追加・取り外しは `statechange` で検出し、接続ごとに 1.2 に定める処理を行います。

同一機種を複数台使う場合、ホストは接続中の MIDI 入出力ポートの組で機器を区別します。抜き差し後も同じ個体を識別する必要がある場合は、機器が対応していれば `DEVICE_ID` を取得して保存できます。

Web MIDI API の利用可否は、OS、ドライバー、ブラウザーの対応と利用者の許可に依存します。

参考文書：Web MIDI API、W3C Working Draft 2025-01-21。

改訂履歴

文書の版	発行日	内容
0.1	2026-10-27	初版（DRAFT）として公開

DRAFT

S-LINK 2.0 機能表

項目	内容
文書の版	0.1（DRAFT）
発行日	2026-10-27
機能表の版（`function_table_revision`）	1

文書の版は、この機能表の文章の改訂を示す番号です。`HELLO` が返す `function_table_revision` とは独立しています。

機能表では、機能番号ごとに単位、データ形式、動作を定めます。 メッセージのバイト構成とホスト・デバイスの動作は、S-LINK 2.0 規格書 が定めます。

入力・出力は、デバイスを基準とします。ホストは入力機能の値を読み取り、出力機能に値を書き込みます。 `access` の `read` は入力機能、`write` は出力機能を表します。

- 機能番号は U14
- 指数はチャンネルごとに `CAPABILITY` の `exponent` が宣言する
- 版は 1。`HELLO` が返す `function_table_revision` はこの値

```
機能    61 件（入力 48 / 出力 13）
```

※ 指数・値域・分解能

指数（例）・値域・分解能の各欄は参考情報です。値域欄には、`payload` 型で表せる範囲または機能上の範囲を示します。機能として受け付ける範囲は、各機能の説明欄で定めます。ホストは送受信する値を、チャンネルごとの `CAPABILITY` の `exponent` で解釈します。 同じ `RESIST` でも、mΩ 計は `-5`、MΩ 計は `-1` を宣言します。

`payload` 列の読み方

この表の `payload` は、`READ` のレスポンスや `WRITE` で扱う値の部分です。符号化後の上限は 61 byte です。メッセージの `payload_length` は、この値に `transaction_id`、`function_id`、`port` の計 5 byte を加えた長さを表します（上限 66 byte）。長さフィールド自体は含めません。

`payload` の読み方、型、指数の適用方法は規格書 6.3を参照してください。

機能番号の範囲

範囲	用途	説明
`0x0100–0x3DFF`	正式な機能	機能表に掲載する機能に割り当てる
`0x3E00–0x3EFF`	非公式の機能番号	機器の開発者が独自の機能に割り当てる
`0x3F00–0x3FFE`	予約	機能には割り当てない
`0x3FFF`	該当なし	機能に対応する番号がないことを示す

非公式の機能番号 `0x3E00–0x3EFF` は、同じ番号が機種ごとに異なる機能を表すことがあります。ホストは対象の接続で得た `model_id` と機器の説明書を確認してから解釈します。別の接続の `model_id` や値の定義を流用しません。

独自機能も `CAPABILITY` で宣言し、既存の `READ` / `WRITE` 形式、データ型、長さ上限、応答と状態管理に従います。値の意味・型・長さ・範囲などは機器の説明書が定めます。

通信手順は、規格書 6.4を参照してください。

生の値

function_id	name	表示名	English	access	kind	単位	payload	指数（例）※	値域 ※	分解能 ※	説明
0x0100	DIGITAL_IN	デジタル入力	Digital input	read	boolean	—	value U7	0	0 / 1	—	端子の電圧が Low か High か。スイッチやデジタル出力センサーの値を読み取る
0x0101	DIGITAL_IN_PU	デジタル入力（プルアップ）	Digital input, pull-up	read	boolean	—	value U7	0	0 / 1	—	内蔵のプルアップ抵抗を有効にして読み取る。スイッチを 2 本の線だけで接続できる
0x0102	ANALOG_IN	アナログ入力	Analog input	read	adc	—	value U28	0	0 – 268,435,455	1 カウント	端子の電圧をアナログ・デジタル（A/D）変換し、A/D 値を返す。自作センサーや、ホスト側で独自に換算する素子に使う

電気・磁気・時間

function_id	name	表示名	English	access	kind	単位	payload	指数（例）※	値域 ※	分解能 ※	説明
0x0200	VOLT	電圧	Voltage	read	quantity	V	value S28	−3	±134,217.727 V	1 mV	その瞬間の電圧。符号つき。直流回路や、READ を繰り返して行う波形観測に使う
0x0201	VOLT_RMS	電圧（実効値）	Root mean square voltage	read	quantity	V	value U28	−3	0 – 268,435.455 V	1 mV	電圧の実効値。交流の測定に使う
0x0202	CURRENT	電流	Current	read	quantity	A	value S28	−6	±134.217727 A	1 µA	その瞬間の電流。符号つき。オームの法則に使う
0x0203	CURRENT_RMS	電流（実効値）	Root mean square current	read	quantity	A	value U28	−6	0 – 268.435455 A	1 µA	電流の実効値。クランプメーターはこれを返す

function_id	name	表示名	English	access	kind	単位	payload	指数（例）※	値域 ※	分解能 ※	説明
0x0204	RESIST	抵抗	Resistance	read	quantity	ohm	value U28	−2	0 – 2,684,354.55 Ω	0.01 Ω	抵抗。電熱線や直列並列の合成抵抗に使う
0x0205	CAP	静電容量	Capacitance	read	quantity	F	value U28	−10	0 – 0.026843545 5 F	100 pF	静電容量。コンデンサーの容量測定や、抵抗を使った放電特性の確認に使う
0x0206	IND	インダクタンス	Inductance	read	quantity	H	value U28	−7	0 – 26.8435455 H	0.1 µH	インダクタンス。自己誘導や、コイルとコンデンサーで構成する共振回路の測定に使う
0x0207	POWER	電力	Power	read	quantity	W	value S28	−3	±134,217.727 W	1 mW	電力。符号つき。発電設備や電気機器の出力測定に使う
0x0208	ENERGY	電力量	Energy	read	quantity	J	value S28	−1	±13,421,772.7 J	0.1 J	電力量。電源投入時を 0 とする累積量。充電で増え、放電で減る
0x0209	EFIELD	電界	Electric field	read	quantity	V/m	value S28	−1	±13,421,772.7 V/m	0.1 V/m	測定軸方向の電界。正の向きは機器の説明書が定める
0x020A	MAG	磁束密度	Magnetic flux density	read	quantity	T	x / y / z S28	−8	±1.34217727 T	10 nT	3 軸の磁束密度。x / y / z のすべてに同じ exponent を適用する
0x020B	FREQ	周波数	Frequency	read	quantity	Hz	value U28	−2	0 – 2,684,354.55 Hz	0.01 Hz	周波数。音の高さや交流の測定に使う
0x020C	GATE_TIME	通過時間	Gate time	read	quantity	s	duration U28 / count U14	−6	0 – 268.435455 s / 0 – 16,383	1 µs	光電ゲートなどで、ゲートが遮られていた時間を測る。READ は最後に終了した測定の duration と累積 count を返す。count は遮られた回数で、exponent を適用しない。電源投入時は 0 とし、遮られるたびに 1 増やす。16,383 の次は 0 に戻る

DRAFT

環境

function_id	name	表示名	English	access	kind	単位	payload	指数（例）※	値域 ※	分解能 ※	説明
0x0300	ILLUM	明るさ	Illuminance	read	quantity	lx	value U28	−2	0 – 2,684,354.55 lx	0.01 lx	人の目の感度で重み付けした明るさ。日なたと日かげの比較に使う
0x0301	UV	紫外線	Ultraviolet index	read	quantity	1	value U14	−1	0 – 1,638.3	0.1	国際照明委員会（CIE）の紅斑作用スペクトルに基づき、国際標準化機構（ISO）の規格 17166 に従って重み付けした紫外線の放射照度を、25 mW/m² で割った値
0x0302	WAVELENGTH	波長	Wavelength	read	quantity	nm	value U14	−1	0 – 1,638.3 nm	0.1 nm	最も強い成分の波長。回折格子を使った発光スペクトルの測定などに使う
0x0303	REFRACT	屈折率	Refractive index	read	quantity	1	value U28	−5	0 – 2,684.35455	0.00001	屈折率。光の屈折のほか、濃度や純度の測定にも使う
0x0304	TEMP	温度	Temperature	read	quantity	degC	value S28	−2	±1,342,177.27 ℃	0.01 ℃	温度
0x0305	HUMID	湿度	Humidity	read	quantity	percentRH	value U14	−2	0 – 163.83 %RH	0.01 %RH	相対湿度（%RH）。気温と組み合わせた環境監視に使う
0x0306	PRESS	気圧	Pressure	read	quantity	hPa	value U28	−2	0 – 2,684,354.55 hPa	0.01 hPa	大気圧。高度による変化や、天気の移り変わりを追う
0x0307	DIFF_PRESS	差圧	Differential pressure	read	quantity	hPa	value S28	−4	±13,421.7727 hPa	0.0001 hPa	2 つの圧力口の圧力差。ベルヌーイの定理やピトー管を使った測定に使う
0x0308	CO2	二酸化炭素	Carbon dioxide	read	quantity	ppm	value U28	0	0 – 268,435,455 ppm	1 ppm	二酸化炭素の濃度。呼吸・燃焼・光合成などに伴う濃度変化の測定に使う
0x0309	O2	酸素	Oxygen	read	quantity	percent	value U14	−2	0 – 163.83 %	0.01 %	酸素の体積分率。燃焼などに伴う酸素濃度の変化を測る
0x030A	PH	pH	pH	read	quantity	1	value U14	−2	0 – 163.83	0.01	水溶液の酸性・アルカリ性を表す。滴定や水質管理などに使う
0x030B	ORP	酸化還元電位	Redox potential	read	quantity	V	value S28	−4	±13,421.7727 V	0.1 mV	参照電極を基準とする電位。標準水素電極への換算の要否は機器の説明書が定める

DRAFT

function_id	name	表示名	English	access	kind	単位	payload	指数（例）※	値域 ※	分解能 ※	説明
0x030C	DO	溶存酸素	Dissolved oxygen	read	quantity	mg/L	value U14	−2	0 – 163.83 mg/L	0.01 mg/L	水に溶けている酸素の量。水質評価や、生物の呼吸・光合成に伴う変化の把握に使う
0x030D	TURB	濁度	Turbidity	read	quantity	FNU	value U14	−1	0 – 1,638.3 FNU	0.1 FNU	860 nm・90° 散乱で測った濁り（国際標準化機構（ISO）の規格 7027-1）。FNU（ホルマジン比濁単位）で表す。ろ過や川の水の観察に使う
0x030E	EC	導電率	Conductivity	read	quantity	S/m	value U28	−4	0 – 26,843.5455 S/m	0.0001 S/m	電気の通しやすさ（導電率）。25 °C 換算かどうかは機器の説明書が定める

力学・運動

function_id	name	表示名	English	access	kind	単位	payload	指数（例）※	値域 ※	分解能 ※	説明
0x0400	DIST	距離	Distance	read	quantity	mm	value U28	−1	0 – 26,843,545.5 mm	0.1 mm	対象までの距離。物体の位置や動きの測定に使う
0x0401	DISP	変位	Displacement	read	quantity	mm	value S28	−2	±1,342,177.27 mm	0.01 mm	基準位置からの符号つきのずれ。単振動やばね振り子に使う
0x0402	SPEED	速度	Velocity	read	quantity	m/s	value S28	−3	±134,217.727 m/s	1 mm/s	測定軸の向きを正とする速度の成分。力学台車や走行ロボットに使う
0x0403	ACCEL	加速度	Acceleration	read	quantity	m/s2	x / y / z S28	−4	±13,421.7727 m/s²	0.0001 m/s²	3 軸の加速度。x / y / z のすべてに同じ exponent を適用する。傾き・振動・落下の測定に使う
0x0404	GYRO	角速度	Angular rate	read	quantity	deg/s	x / y / z S28	−2	±1,342,177.27 deg/s	0.01 deg/s	3 軸の角速度。x / y / z のすべてに同じ exponent を適用する。回転運動やロボットの姿勢に使う
0x0405	ANG_ACCEL	角加速度	Angular acceleration	read	quantity	deg/s2	x / y / z S28	−1	±13,421,772.7 deg/s²	0.1 deg/s²	3 軸の角加速度。x / y / z のすべてに同じ exponent を適用する。回転速度の変化や、回転運動の解析に使う
0x0406	FORCE	力	Force	read	quantity	N	value S28	−3	±134,217.727 N	1 mN	力。符号つき。ばねの伸び、浮力、摩擦力などの測定に使う
0x0407	TORQUE	トルク	Torque	read	quantity	N·m	value S28	−4	±13,421.7727 N·m	0.1 mN·m	回転させる力。回転体のつり合いや慣性モーメントの評価に使う

DRAFT

function_id	name	表示名	English	access	kind	単位	payload	指数（例）※	値域 ※	分解能 ※	説明
0x0408	STRAIN	ひずみ	Strain	read	quantity	1	value S28	−7	±13.4217727	0.0000001	ひずみ（ΔL/L）。材料の弾性や変形の評価に使う
0x0409	MASS	質量	Mass	read	quantity	kg	value S28	−6	±134.217727 kg	1 mg	質量。符号つき。反応や溶解に伴う質量変化の測定に使う
0x040A	ENCODER	回転量	Rotation	read	quantity	deg	value S28	−1	±13,421,772.7 deg	0.1 deg	軸の回転量。電源投入時を 0 とする累積量。負で逆回転

検知・操作・生体

function_id	name	表示名	English	access	kind	単位	payload	指数（例）※	値域 ※	分解能 ※	説明
0x0500	BUTTON	ボタン	Button	read	boolean	—	value U7	0	0 / 1	—	ボタンが押されているか。負論理の配線でも 1 が「押している」
0x0501	POT	つまみの位置	Potentiometer	read	quantity	percent	value U14	−2	0 – 100 %	0.01 %	つまみの位置。一方の端が 0 %、もう一方の端が 100 %
0x0502	PIR	人の検知	Passive infrared motion detection	read	boolean	—	value U7	0	0 / 1	—	人が動いたかどうか。焦電センサーの出力をそのまま返す
0x0503	PROXIMITY	近接	Proximity	read	boolean	—	value U7	0	0 / 1	—	物体に触れずに、その有無を検出する。自動ドアなどに使う
0x0504	MAGNET_NEAR	磁気近接	Magnet proximity	read	boolean	—	value U7	0	0 / 1	—	磁石が近いことを検知する。リードスイッチを使った開閉検出や、磁石の性質の観察に使う
0x0505	HEART	心拍	Heart rate	read	quantity	bpm	value U14	0	0 – 16,383 bpm	1 bpm	心拍数

DRAFT

出力機能

function_id	name	表示名	English	access	kind	単位	payload	指数（例）※	値域 ※	分解能 ※	説明
0x1000	LEVEL	出力の強さ	Output level	write	quantity	percent	value U14	−2	0 – 100 %	0.01 %	端子の出力を 0〜100 % で駆動する。on/off も 0 % と 100 % で表す
0x1001	SERVO	サーボモーター	Servo motor	write	quantity	deg	angle U14	−1	0 – 180 deg	0.1 deg	サーボモーターの軸を指定した角度（0〜180 度）に動かす
0x1002	VOLT_OUT	電圧を出す	Voltage output	write	quantity	V	value S28	−3	±134,217.727 V	1 mV	指定した電圧を出す。計装の 0–10 V や D/A 変換に使う
0x1003	RELAY	制御スイッチ（リレー）	Relay	write	boolean	—	value U7	0	0 / 1	—	接点を開閉する
0x1004	MOTOR	モーター	Motor	write	quantity	percent	value S28	−2	−100 – 100 %	0.01 %	出力を −100〜100 % で駆動する。負で逆転
0x1005	PUMP	ポンプ	Pump	write	quantity	percent	value U14	−2	0 – 100 %	0.01 %	出力を 0〜100 % で駆動する
0x1006	FAN	送風	Fan	write	quantity	percent	value U14	−2	0 – 100 %	0.01 %	出力を 0〜100 % で駆動する
0x1007	VIBRATOR	振動	Vibration	write	quantity	percent	value U14	−2	0 – 100 %	0.01 %	出力を 0〜100 % で駆動する
0x1008	ELECTROMAG	電磁石	Electromagnet	write	quantity	percent	value U14	−2	0 – 100 %	0.01 %	出力を 0〜100 % で駆動する

DRAFT

function_id	name	表示名	English	access	kind	単位	payload	指数（例）※	値域 ※	分解能 ※	説明
0x1009	SOLENOID	押し出し	Solenoid	write	boolean	—	value U7	0	0 / 1	—	0 = 戻す、1 = 押す
0x100A	STEPPER	ステッピングモーター	Stepper	write	quantity	deg	angle S28 / speed U14	−1	角度 ±13,421,772.7 deg / 速度 0 – 16,383 deg/s	0.1 deg	目標角度と、その位置まで移動する速度。原点は電源投入時の位置。PORT_RELEASE でそのときの位置が新しい原点になる。speed は deg/s の整数で、exponent は適用しない。0 はデバイスの既定の速さ
0x100B	RGB	フルカラー LED	RGB LED	write	color	—	index U14 / r / g / b U7	0	index 0 – 16,383、各成分 0 – 127	1	index 番目の LED の赤（R）・緑（G）・青（B）を指定する。各成分は 7 bit で、127 が最大値。0–255 の RGB 値を使う場合は、各成分を 1 bit 右へずらす（255 → 127）。LED 数以上の index には ERROR(BAD_REQUEST) を返す
0x100C	HEATER	加熱・冷却	Heater / cooler	write	quantity	percent	value S28	−2	−100 – 100 %	0.01 %	出力を −100〜100 % で駆動する。負の値はペルチェ素子による冷却を表す

boolean の値の意味

function_id	0	1
0x0100 DIGITAL_IN / 0x0101 DIGITAL_IN_PU	Low	High
0x0500 BUTTON	離している	押している
0x0502 PIR	検知なし	検知あり
0x0503 PROXIMITY	物なし	物あり
0x0504 MAGNET_NEAR	磁石なし	磁石あり
0x1003 RELAY	開く	閉じる
0x1009 SOLENOID	戻す	押す

DRAFT